Unidad 1. Fundamentos de la programación

Índice

1. Conceptos básicos de programación 3
   1.1 Definición de programación 3
   1.2 Objetivos y propósito de un programa informático 3
   1.3 Proceso general de desarrollo de software 5
2. Datos, algoritmos y programas 7
   2.1 Concepto de dato y representación de la información 7
   2.2 Tipos de datos (visión general) 8
   2.3 Definición y propiedades de un algoritmo 9
   2.4 Estructuras básicas de un algoritmo 10
   2.5 Representación de algoritmos: pseudocódigo y diagramas de flujo 11
   2.6 Relación entre algoritmo y programa 13
3. Paradigmas de programación 14
   3.1 Concepto de paradigma 15
   3.2 Paradigma imperativo 16
   3.3 Paradigma estructurado 17
   3.4 Paradigma orientado a objetos (POO) 18
   3.5 Paradigma funcional 19
   3.6 Comparación entre paradigmas 20
4. Lenguajes de programación 22
   4.1 Definición de lenguaje de programación 22
   4.2 Clasificación de lenguajes: bajo nivel vs alto nivel 22
   4.3 Lenguajes compilados vs interpretados 24
   4.4 Traducción del código: compilación, interpretación y máquina virtual 25
   4.5 Características generales de los lenguajes modernos 27
5. Herramientas y entornos para el desarrollo de programas 28
   5.1 Editores de texto para programación 28
   5.2 Entornos de desarrollo integrados (IDE) 29
   5.3 Compiladores e intérpretes 30
   5.4 Depuradores (Debuggers) 31
   5.5 Herramientas de análisis y diagnóstico 33
   5.6 Control de versiones 34
   5.7 Gestión del proyecto y automatización (build) 36
6. Errores y calidad de los programas 37

6.1 Tipos de errores 37
6.2 Detección y corrección de errores 38
6.3 Calidad del código: legibilidad, modularidad y mantenibilidad 40
6.4 Introducción al aseguramiento de la calidad 41
Resumen 43

1. Conceptos básicos de programación

1.1 Definición de programación

La programación es el proceso de diseñar, codificar, depurar y mantener el código fuente de programas computacionales. En términos más sencillos, es el arte y la técnica de dar instrucciones a una computadora para que realice tareas específicas.

Una definición formal implica:

- **Comunicación:** Es el medio por el cual los humanos se comunican con las máquinas.
- **Traducción:** Implica traducir la lógica humana y la solución de un problema a un lenguaje que la máquina pueda entender y ejecutar (lenguaje de programación).
- **Secuenciación:** Consiste en crear una secuencia ordenada de pasos (algoritmo) que conducen a un resultado deseado.

Nota: Aunque escribimos en lenguajes de alto nivel (más cercanos al inglés humano), la computadora finalmente procesa estas órdenes en código binario (ceros y unos).

1.2 Objetivos y propósito de un programa informático

Aunque a menudo pensamos en la programación como "escribir código", el código es solo el medio. El propósito real de cualquier programa informático se puede desglosar en cuatro pilares fundamentales que definen su utilidad en el mundo real.

A. El Modelo de Procesamiento de Información (Entrada-Proceso-Salida)

El objetivo más básico de un programa es transformar datos en información útil. Todo software, desde una calculadora simple hasta un sistema bancario, sigue el **ciclo IPO** (*Input-Process-Output*).

1. **Entrada (Input):** El programa debe tener mecanismos para recibir datos "crudos" del mundo exterior.

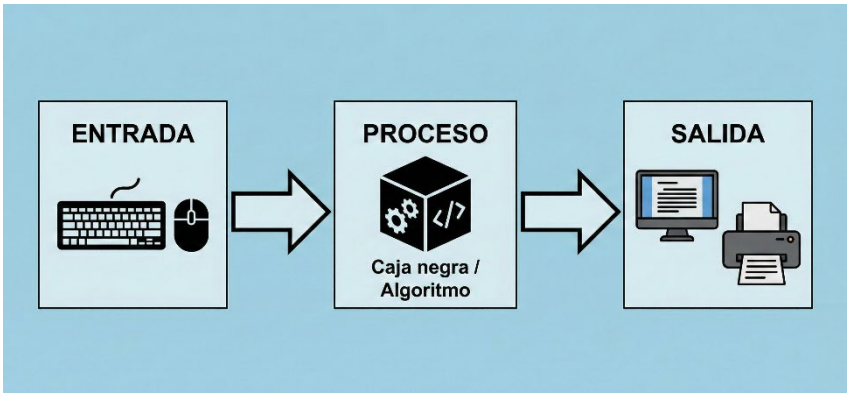
    - *Ejemplos:* Texto introducido por teclado, clics del ratón, lectura de un fichero, o datos recibidos desde una base de datos.

2. **Proceso (Algoritmo):** Es el corazón del programa. Aquí se aplican las operaciones lógicas y matemáticas sobre los datos recibidos.

    - El programa utiliza **variables** para almacenar temporalmente estos datos en la memoria y **operadores** para manipularlos.

3. **Salida (Output):** El resultado del procesamiento se devuelve al usuario o a otro sistema.

    - *Ejemplos:* Mostrar información en pantalla, guardar un archivo modificado o enviar un comando a una impresora.



B. Automatización de Tareas Repetitivas

Uno de los propósitos principales de la informática es liberar al ser humano de tareas tediosas. Los ordenadores son excelentes realizando la misma tarea millones de veces sin cansarse ni cometer errores por fatiga.

- **Eficiencia:** Un programa puede procesar en segundos lo que a un humano le llevaría días.
- **Estructuras de Repetición:** Para lograr esto, los programas utilizan estructuras de control (bucles) que permiten ejecutar un bloque de instrucciones tantas veces como sea necesario[6].
- *Ejemplo:* Un programa que renombra automáticamente 1.000 fotos o calcula la nómina de 500 empleados con un solo clic.

C. Modelado y Abstracción de la Realidad

Especialmente en la **Programación Orientada a Objetos** (que es el enfoque principal de este módulo), el propósito de un programa es representar o "modelar" objetos de la vida real dentro del ordenador.

- Un programa de gestión de biblioteca no solo "guarda textos", sino que crea representaciones digitales de "Libros", "Socios" y "Préstamos".
- Cada uno de estos elementos tiene sus propias características (atributos) y comportamientos (métodos).
- El objetivo aquí es crear un sistema que refleje la lógica del negocio o del problema real que se intenta resolver.

D. Persistencia y Gestión de Datos

Un programa no solo procesa datos en el momento, sino que tiene el objetivo de garantizar que la información perdure en el tiempo.

- **Volatilidad vs. Persistencia:** Mientras el programa se ejecuta, los datos viven en la memoria RAM (volátil). Uno de los propósitos críticos del software es trasladar esos datos a medios de almacenamiento permanente (discos duros, bases de datos).
- Esto permite recuperar la información días o años después para su análisis o modificación.

Podemos resumir los objetivos en este esquema funcional:

Objetivo	Descripción	Elemento de programación asociado
Interactuar	Recibir y mostrar datos.	Entrada/Salida (I/O)
Recordar	Almacenar datos temporalmente.	Variables y Estructuras de datos 14141414
Decidir	Tomar caminos distintos según los datos.	Estructuras de selección (If/Else)
Repetir	Ejecutar tareas múltiples veces.	Estructuras de repetición (Bucles)
Persistir	Guardar datos para el futuro.	Ficheros y Bases de Datos

1.3 Proceso general de desarrollo de software

El desarrollo de software es un proceso sistemático que busca transformar una necesidad o problema en una solución informática funcional y de calidad. Aunque existen diversas metodologías, las etapas fundamentales son universales:

1. Análisis de requisitos (¿Qué vamos a hacer?)

Antes de escribir una sola línea de código, es necesario comprender el problema. En esta fase se define el **"qué"**: qué debe hacer el programa y qué restricciones tiene.

- **Identificación del problema:** Se determina el objetivo del software (como vimos en el punto 1.2).
- **Especificación de requisitos:** Se definen las entradas necesarias (datos) y las salidas esperadas.
- *Vinculación curricular:* Esta fase es necesaria para que el alumno pueda "reconocer la estructura de un programa informático" e identificar los bloques que lo componen.

2. Diseño del sistema (¿Cómo lo vamos a hacer?)

Es la planificación de la solución. Aquí se define la lógica sin usar todavía un lenguaje de programación específico.

- **Algoritmia:** Se crean los pasos lógicos para resolver el problema (pseudocódigo o diagramas de flujo, que veremos en el punto 2).
- **Estructura de datos:** Se decide cómo se organizará la información (variables, tipos de datos).
- **Diseño modular:** Se divide el problema grande en problemas más pequeños (clases, objetos, métodos), aplicando los principios de la programación orientada a objetos.

3. Codificación o Implementación (Construcción)

Es la fase de traducción. El diseño realizado en la fase anterior se traduce a un lenguaje de programación específico (en este módulo, un lenguaje orientado a objetos).

- **Escritura de código fuente:** Uso de la sintaxis del lenguaje, palabras reservadas y estructuras de control.
- **Uso de entornos de desarrollo (IDE):** Se utilizan herramientas profesionales para escribir y compilar el programa.
- *Objetivo:* Obtener un programa ejecutable.

4. Pruebas y Depuración (Testing & Debugging)

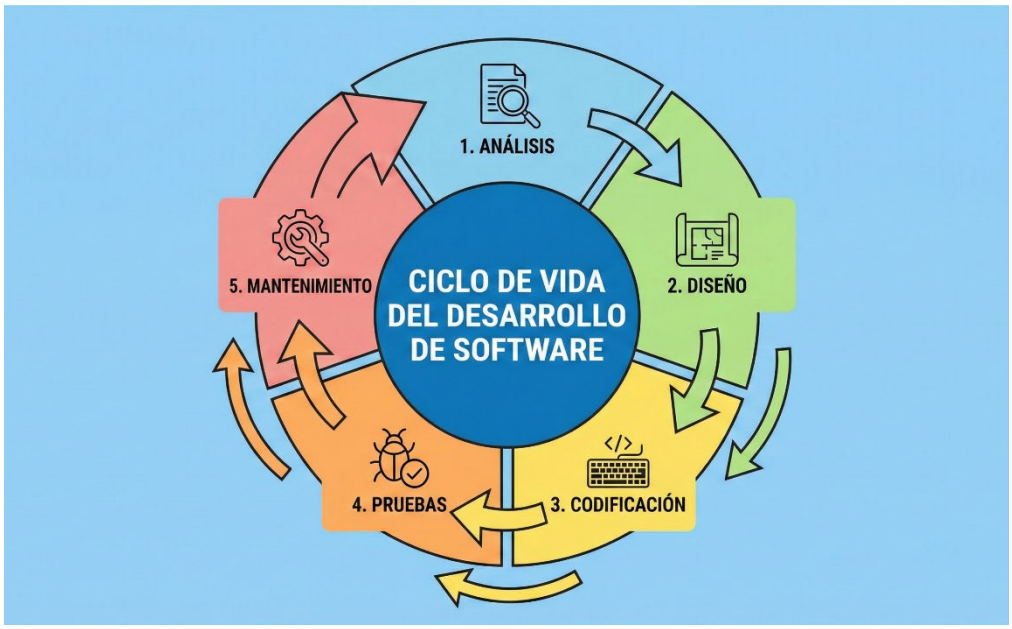
Es imposible desarrollar software libre de errores en el primer intento. Esta fase se centra en garantizar que el programa funcione correctamente y sea fiable.

- **Pruebas (Testing):** Ejecutar el programa con diferentes datos de entrada para verificar si los resultados son los esperados.
- **Depuración (Debugging):** Proceso de identificar, localizar y corregir errores (bugs) en el código. Esto incluye errores de sintaxis, ejecución y lógica.
- *Vinculación curricular:* El currículo exige explícitamente que "se han probado y depurado los programas" y "se ha escrito y depurado código".

5. Documentación y Mantenimiento

Un programa profesional debe ser comprensible para otros programadores y adaptable en el tiempo.

- **Documentación interna:** Inserción de comentarios en el propio código fuente para explicar partes complejas de la lógica. El criterio de evaluación indica que se debe "comentar y documentar el código".
- **Mantenimiento:** Implica actualizar el software para corregir fallos detectados tras su lanzamiento, mejorar el rendimiento o adaptar el programa a nuevos requisitos o sistemas operativos.



2. Datos, algoritmos y programas

Para que un ordenador pueda resolver un problema, necesitamos dos cosas: la información que debe procesar y los pasos necesarios para hacerlo.

2.1 Concepto de dato y representación de la información

En el mundo de la programación, es crucial distinguir entre lo que la máquina recibe y lo que la máquina "entiende" o produce.

A. Diferencia entre Dato e Información

Aunque en el lenguaje cotidiano usamos estos términos como sinónimos, en informática tienen matices distintos:

- **Dato:** Es la unidad mínima de conocimiento o un hecho aislado que, por sí solo, no tiene un significado completo para los humanos. Es la "materia prima".
    - *Ejemplo:* El número `25`, la palabra `"Azul"`, o la fecha `12/05/2024`. Si ves un `25` escrito en un papel, no sabes si es una edad, una temperatura o un precio.
- **Información:** Es un conjunto de datos procesados, ordenados y organizados dentro de un contexto que les da significado y utilidad.
    - *Ejemplo:* "La temperatura actual es de **25** grados". Aquí, el dato `25` ya tiene contexto y se convierte en información útil para tomar decisiones (por ejemplo, encender el aire acondicionado).

La ecuación fundamental de la informática:

Datos + Proceso (Contexto) = Información

B. Representación de la información en el ordenador

Los seres humanos utilizamos lenguajes complejos, alfabetos y sistema decimal (base 10) para representar nuestra realidad. Sin embargo, el ordenador es una máquina electrónica que, en su nivel más bajo (hardware), solo entiende de estados eléctricos: encendido/apagado, pasa corriente/no pasa corriente.

Para que un programa funcione, cualquier dato (texto, imagen, sonido, número) debe ser **codificado** a un formato que la máquina pueda almacenar.

1. El Sistema Binario:

Es el lenguaje nativo del ordenador. Toda la información se representa utilizando únicamente dos símbolos: 0 y 1.

- **Bit (Binary Digit):** Es la unidad mínima de información. Puede valer solo `0` o `1`.
- **Byte:** Es la agrupación estándar de 8 bits (ej. `10110010`). Es la unidad básica de almacenamiento que suele representar un carácter (una letra, un número).

2. Codificación de datos:

Para que nosotros veamos una "A" en la pantalla, el ordenador realiza una conversión o codificación:

- **Texto:** Se usan estándares como **ASCII** o **Unicode**. Por ejemplo, en ASCII, la letra 'A' se guarda internamente como el número `65` (que en binario es `01000001`).
- **Números:** Se transforman directamente mediante conversión matemática de decimal a binario.

2.2 Tipos de datos (visión general)

En programación, no basta con tener un dato; el ordenador necesita saber **de qué naturaleza** es ese dato para saber cómo procesarlo y cuánto espacio de memoria reservar para él. Esta clasificación es lo que conocemos como **tipos de datos**.

La elección correcta del tipo de dato es fundamental, ya que define qué operaciones se pueden realizar (por ejemplo, se pueden sumar números, pero no se pueden sumar "fechas" con "nombres" de forma aritmética estándar).

Podemos clasificar los tipos de datos en tres grandes categorías fundamentales:

1. Tipos de Datos Básicos (Primitivos)

Son los tipos de información más elementales, aquellos que el lenguaje de programación ofrece de forma nativa y que no están compuestos por otros datos. Se corresponden directamente con los literales básicos del lenguaje.

- **Numéricos enteros:** Representan números sin parte decimal. Se utilizan para contar (ej. `5`, `-10`, `0`).
- **Numéricos reales (coma flotante):** Representan números con decimales. Son necesarios para cálculos de precisión (ej. `3.14`, `9.99`, `-0.05`).

- **Carácter (Char):** Representan un único símbolo alfanumérico, letra o signo (ej. `'A'`, `'z'`, `'@'`, `'9'`).
- **Lógicos (Booleanos):** Solo pueden tener dos valores posibles: **Verdadero** (True) o **Falso** (False). Son esenciales para la toma de decisiones en las estructuras de control.

2. Tipos de Datos Estructurados (Compuestos)

Surgen de la necesidad de gestionar colecciones de datos o agrupar información relacionada. Se construyen a partir de los tipos básicos.

- **Cadenas de caracteres (Strings):** Es una secuencia ordenada de caracteres que forma texto (ej. "Hola Mundo"). En muchos lenguajes, se tratan como objetos especiales debido a su importancia.
- **Arrays (Arreglos/Matrices):** Son estructuras que almacenan una colección de datos del **mismo tipo** bajo un mismo nombre. Se accede a cada elemento mediante un índice numérico.
    - *Ejemplo:* Una lista de las notas de 20 alumnos.

3. Tipos de Datos Abstractos y Objetos

Dado que este módulo se centra en la **Programación Orientada a Objetos**, este es el tipo de dato más relevante.

- **Clases y Objetos:** Permiten al programador definir sus propios tipos de datos complejos que modelan la realidad. Un objeto agrupa tanto los datos (atributos) como las operaciones que se pueden hacer con ellos (métodos).
    - *Ejemplo:* Un tipo de dato `Coche` que contiene la marca (texto), la velocidad (número) y métodos como `acelerar()`.
- **Colecciones:** Estructuras dinámicas más avanzadas que los arrays, como listas, pilas o colas, que permiten almacenar y manipular grupos de objetos de forma eficiente.

Más info sobre el tipado: Dependiendo del lenguaje, el tipo de dato debe definirse explícitamente al escribir el código (tipado estático) o se deduce automáticamente durante la ejecución (tipado dinámico). En este módulo trabajaremos con conversiones de tipo explícitas e implícitas (casting) para transformar datos de un tipo a otro.

2.3 Definición y propiedades de un algoritmo

Antes de intentar hablar con el ordenador mediante un lenguaje de programación, debemos tener claro qué queremos decirle. Aquí es donde entra el algoritmo.

A. Definición de Algoritmo

Un algoritmo es un **conjunto ordenado y finito de operaciones sistemáticas** que permite hallar la solución a un problema.

En términos sencillos, es una "receta" paso a paso. No es código informático todavía, es la lógica pura de la solución. Si el algoritmo está mal planteado, el programa fallará, sin importar lo bien que escribas el código.

Analogía clásica: Una receta de cocina es un algoritmo.

- **Entrada:** Ingredientes (huevos, aceite, sal).
- **Proceso:** Pasos secuenciales (batir, calentar sartén, echar mezcla).
- **Salida:** Tortilla francesa.

B. Propiedades fundamentales

Para que una secuencia de pasos sea considerada un algoritmo válido en informática, debe cumplir rigurosamente con cinco características o propiedades básicas:

1. **Preciso:** Cada paso debe estar claramente indicado y ser inequívoco. El orden de ejecución de las instrucciones es vital. No podemos decir "mezcla los ingredientes", debemos decir "añade primero la harina y después el agua".
2. **Definido** (Determinista): Si se sigue el algoritmo dos veces con los mismos datos de entrada, se debe obtener invariablemente el mismo resultado. No puede haber lugar al azar o a la interpretación subjetiva de la máquina.
3. **Finito:** Todo algoritmo debe tener un principio y un fin. Por muy compleja que sea la tarea, debe terminar en algún momento tras un número determinado de pasos. Un algoritmo que se queda procesando eternamente (bucle infinito) es un algoritmo erróneo.
4. **Entrada (Input)**: El algoritmo puede requerir de cero o más elementos de entrada (datos) que provienen del exterior para trabajar con ellos.
5. **Salida (Output)**: Debe producir al menos un resultado. Esta salida es la solución al problema planteado y tiene una relación directa con las entradas procesadas.

Importancia en el desarrollo

Como vimos en el ciclo de vida del software, el diseño del algoritmo ocurre **antes** de la codificación. Un error en esta etapa es mucho más costoso de corregir que un simple error de sintaxis en el código.

2.4 Estructuras básicas de un algoritmo

Cualquier algoritmo, por muy complejo que sea (desde sumar dos números hasta la inteligencia artificial de un videojuego), se construye combinando únicamente tres tipos de estructuras de control básicas[1].

Estas estructuras determinan el orden en el que se ejecutan las instrucciones, lo que conocemos como **flujo de control**[2].

1. Estructura Secuencial

Es la estructura más simple y la que ocurre por defecto. Las instrucciones se ejecutan una tras otra, siguiendo el orden en el que han sido escritas, de arriba hacia abajo. La salida de una instrucción es la entrada de la siguiente.

- **Lógica:** Paso 1 → Paso 2 → Paso 3.
- *Ejemplo:*

1. Leer la base.
2. Leer la altura.
3. Calcular área (base * altura / 2).
4. Mostrar resultado.

2. Estructuras Selectivas (Condicionales)

Permiten que el algoritmo tome decisiones y elija entre distintos caminos en función de si se cumple o no una condición (generalmente una expresión lógica que da como resultado Verdadero o Falso). Rompen la linealidad del programa.

- **Simple (Si... entonces):** Ejecuta una acción solo si la condición es cierta.
- **Doble (Si... entonces... sino):** Si la condición es cierta realiza la acción A; si es falsa, realiza la acción B.
- **Múltiple:** Permite elegir entre varias opciones posibles (se suele implementar con estructuras tipo `switch` o `case`).

3. Estructuras Iterativas (Repetitivas o Bucles)

Permiten ejecutar un bloque de instrucciones múltiples veces de forma automática, sin tener que escribir el código repetido. La repetición se mantiene mientras se cumpla una condición determinada.

- **Bucle "Mientras" (While):** Evalúa la condición al principio. Si es cierta, ejecuta y vuelve a comprobar.
- **Bucle "Hacer... Mientras" (Do-While):** Ejecuta al menos una vez y luego comprueba la condición.
- **Bucle "Para" (For):** Repite un número determinado y conocido de veces (ideal para recorrer arrays o contar).

2.5 Representación de algoritmos: pseudocódigo y diagramas de flujo

Una vez que tenemos la solución en nuestra cabeza (el algoritmo), necesitamos plasmarla en un formato que sirva de puente entre el lenguaje humano y el lenguaje de programación (Java). Para ello existen dos herramientas fundamentales[1].

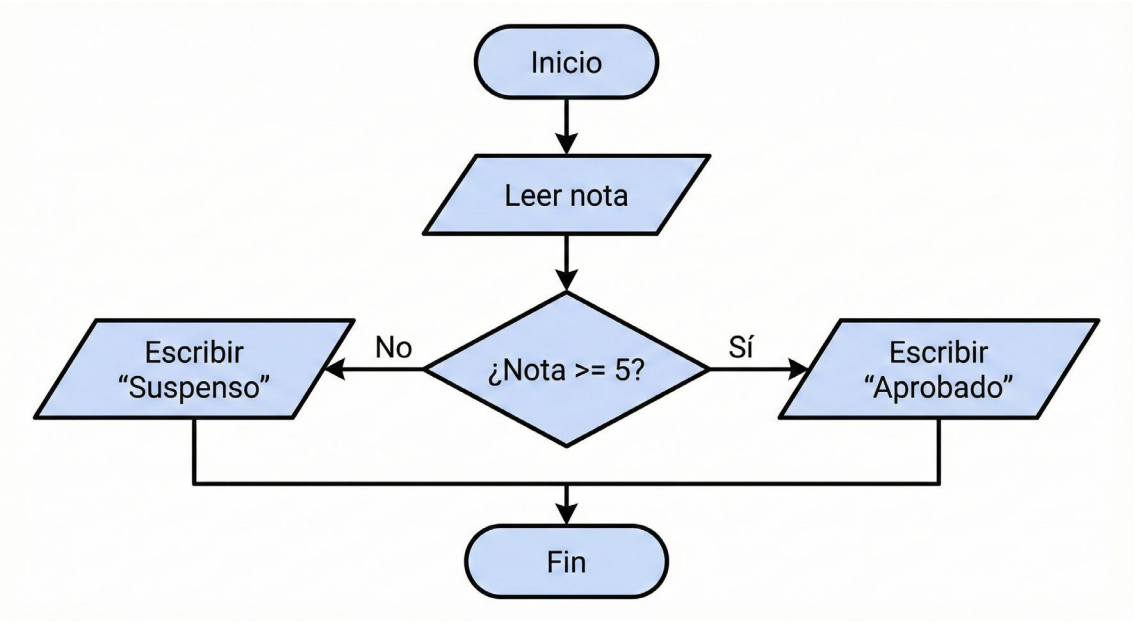
A. Diagramas de Flujo (Flowcharts)

Es una representación **gráfica** del algoritmo. Utiliza símbolos estándar normalizados (generalmente ANSI/ISO) unidos por flechas que indican la secuencia de ejecución (el flujo).

Son muy útiles para entender la lógica global de un vistazo, pero pueden volverse desordenados en programas muy grandes.

Simbología básica:

Símbolo	Nombre	Función
Óvalo	Inicio / Fin	Marca el punto de partida y final del algoritmo.
Paralelogramo	Entrada / Salida	Lectura de datos o impresión de resultados.
Rectángulo	Proceso	Operaciones, cálculos o asignaciones.
Rombo	Decisión	Pregunta que divide el flujo en SÍ o NO.



B. Pseudocódigo

Es una representación **narrativa** del algoritmo que utiliza un lenguaje mixto: palabras clave en nuestro idioma (o en inglés) y estructura de programación real, pero sin la rigidez de la sintaxis de un lenguaje específico como Java.

Es la herramienta favorita de los programadores porque:

1. Es fácil de escribir y corregir.
2. Se traduce casi línea por línea al código final.
3. Permite centrarse en la **lógica** sin preocuparse por los puntos y comas.

2.6 Relación entre algoritmo y programa

A menudo, los estudiantes principiantes confunden estos dos términos o piensan que son lo mismo. Sin embargo, entender la distinción y la relación entre ellos es clave para ser un buen desarrollador.

A. La Distinción Fundamental

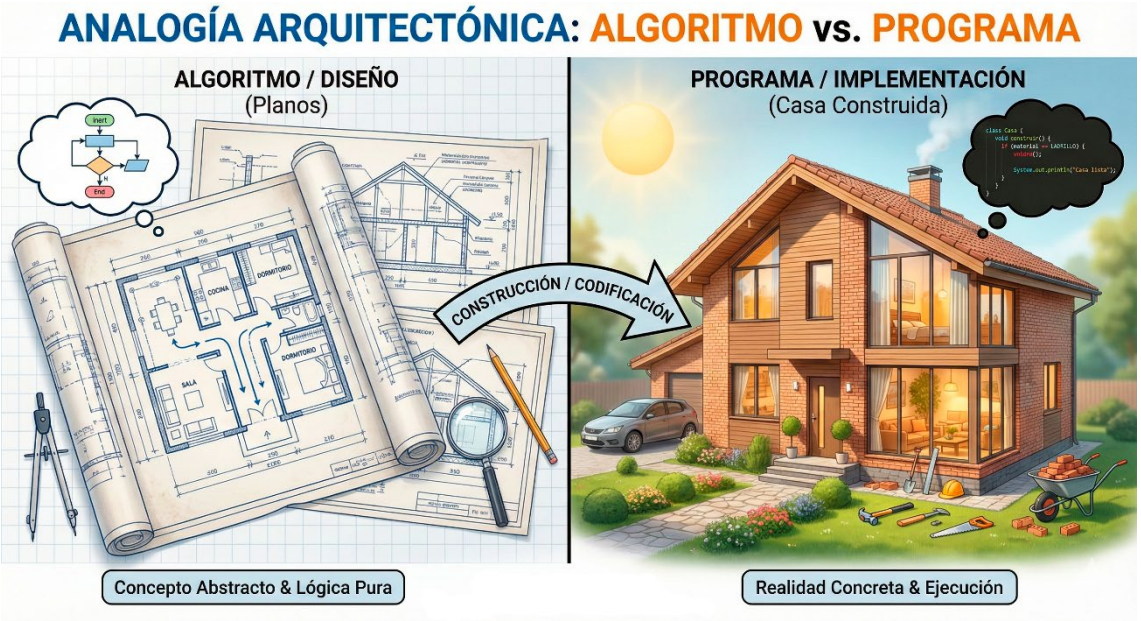
Podemos definir la relación de la siguiente manera:

- **El Algoritmo es el diseño (el "qué" y el "cómo" lógico).** Es una entidad abstracta, independiente de la máquina y del lenguaje. Existe en tu mente o en un papel.
- **El Programa es la implementación (la construcción real).** Es la traducción de ese algoritmo a un lenguaje concreto que una computadora específica puede entender y ejecutar.

Analogía de la Arquitectura:

- **El Algoritmo** son los **planos** del arquitecto. Definen dónde van las paredes y las puertas, pero no son habitables.
- **El Programa** es la **casa construida**. Es la materialización de esos planos usando ladrillos y cemento (código).

Más info*:* Con unos mismos planos (algoritmo), puedes construir la casa usando madera, ladrillo u hormigón (diferentes lenguajes de programación como Java, Python o C++), pero la distribución de la casa será idéntica.



B. Del Algoritmo al Programa: La Fase de Codificación

El proceso de convertir un algoritmo en un programa se llama **codificación** o implementación. En este paso, el programador debe adaptar la lógica general a las reglas estrictas (sintaxis) del lenguaje elegido.

Característica	Algoritmo (Pseudocódigo)	Programa (Java)
Lenguaje	Humano / Natural estructurado	Formal / Artificial (Java)
Rigidez	Flexible (puedes decir "Escribir" o "Mostrar")	Estricta (`System.out.println("");`)
Ejecución	No ejecutable por la máquina	Ejecutable por la JVM (Java Virtual Machine)
Errores	Solo lógicos	Lógicos y de Sintaxis

C. Independencia del Lenguaje

Una de las propiedades más importantes de un buen algoritmo es que debe ser **independiente del lenguaje de programación**.

Si diseñas bien el algoritmo para "Calcular el área de un círculo", ese mismo diseño debe servir para programarlo en Java hoy, en C# mañana o en un lenguaje que aún no se ha inventado dentro de 10 años. Si tu algoritmo depende de una característica exclusiva de Java, entonces no es un algoritmo puro, es ya un pre-código.

Ejemplo de Traducción (Mapping)

Retomando el ejemplo del punto anterior, veamos la traducción directa:

1. Algoritmo (Lógica pura):

Si la nota es mayor o igual a 5, entonces aprobar.

2. Programa (Sintaxis Java):
Java
```
if (nota >= 5) {
    System.out.println("Aprobado");
}
```

3. Paradigmas de programación

Hasta ahora hemos visto que un algoritmo es una secuencia de pasos. Sin embargo, cuando el software crece, no podemos tener simplemente una lista gigante de millones de líneas de código. Necesitamos una filosofía o un "estilo" para organizarlo. Eso es un paradigma.

3.1 Concepto de paradigma

La palabra **paradigma** proviene del griego *paradeigma*, que significa "modelo" o "ejemplo". En el contexto de la ingeniería de software y la informática, un paradigma de programación es un **enfoque fundamental, estilo o filosofía para la construcción de software**.

No se trata de un lenguaje de programación específico, sino de la **forma de pensar** y resolver problemas que adopta el programador.

¿Qué define a un paradigma?

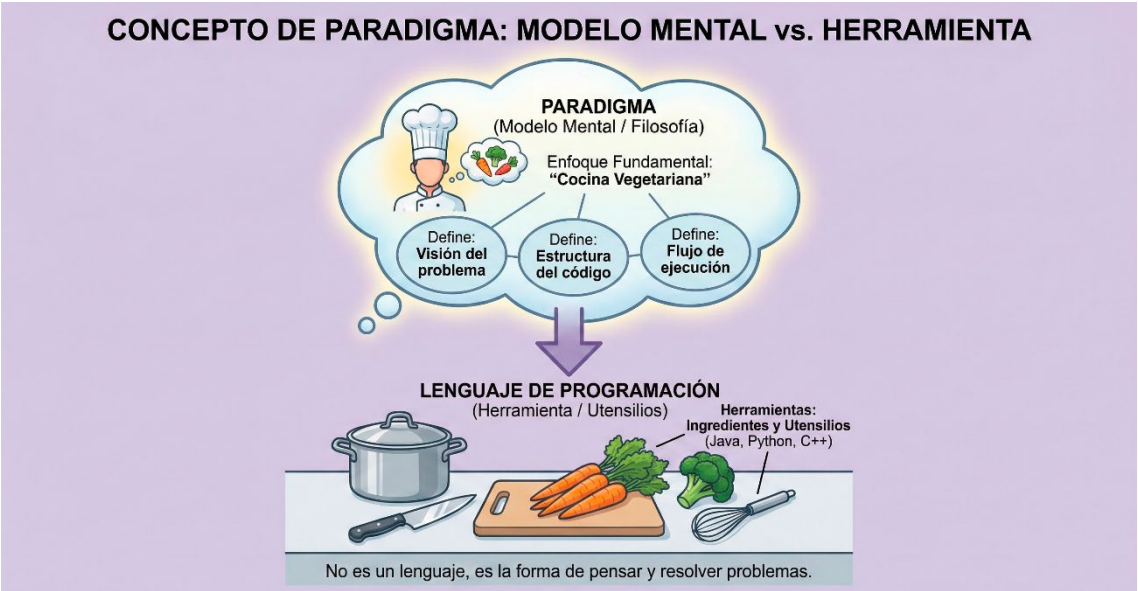
Un paradigma establece las reglas del juego sobre cómo se estructura y ejecuta el código. Define:

1. **La visión del problema:** ¿Vemos el programa como una lista de órdenes (hacer esto, luego aquello) o como un conjunto de objetos que interactúan?
2. **La estructura del código:** ¿Cómo agrupamos las instrucciones? (en funciones, en clases, en bloques lógicos).
3. **El flujo de ejecución:** ¿Cómo se controla el orden en que suceden las cosas?

Paradigma vs. Lenguaje de Programación

Es vital distinguir entre el estilo (paradigma) y la herramienta (lenguaje).

- **El Paradigma** es el modelo mental (ej. "Cocina vegetariana").
- **El Lenguaje** es la herramienta (ej. "Los ingredientes y utensilios").



Más info: Muchos lenguajes modernos son **multiparadigma**. Java, el lenguaje de este curso, es principalmente **Orientado a Objetos**, pero permite usar estilo **Imperativo** (dentro de los métodos) y recientemente ha incorporado características del paradigma **Funcional**.

¿Por qué existen varios?

No existe un "mejor paradigma" absoluto. Cada uno surgió para solucionar las limitaciones del anterior o para resolver tipos de problemas específicos de manera más eficiente:

- Para cálculos matemáticos puros, el paradigma funcional suele ser superior.
- Para sistemas complejos de gestión (como un banco o un videojuego), el paradigma orientado a objetos es el estándar industrial.

3.2 Paradigma imperativo

El paradigma imperativo es el enfoque más antiguo y tradicional de la programación. Su nombre proviene del latín *imperare* (mandar o dar órdenes).

Concepto Fundamental

En este paradigma, un programa se concibe como una **secuencia de instrucciones o comandos** que el ordenador debe ejecutar paso a paso para alcanzar un resultado. El programador dicta al ordenador **cómo** debe realizar la tarea, detallando cada cambio de estado en la memoria.

Se basa directamente en la arquitectura física de los ordenadores (Arquitectura de von Neumann):

1. Tenemos una memoria donde se guardan datos (variables).
2. Tenemos un procesador que ejecuta instrucciones que leen o modifican esos datos.

Características Clave

Para identificar si estamos programando de forma imperativa, debemos fijarnos en tres elementos:

- **Estado Mutable:** El programa tiene un "estado" actual definido por el valor de sus variables en ese momento. Las instrucciones cambian estos valores constantemente.
    - *Ejemplo:* `x = 5` (estado inicial) → `x = x + 1` (nuevo estado: x vale 6).
- **Sentencias de Asignación:** Es la operación principal. Asignar un valor a una variable es la forma de "recordar" un resultado intermedio.
- **Orden de Ejecución:** El orden en que escribimos las líneas es crucial. Si cambiamos el orden de dos instrucciones que modifican la misma variable, el resultado final cambia.

  Analogía: Imagina a un entrenador dando instrucciones a un atleta:

  1. "Levanta el pie derecho."
  2. "Da un paso adelante."
  3. "Baja el pie."

  El entrenador está usando un **estilo imperativo**: da órdenes precisas que cambian el estado del atleta paso a paso.

Relación con Java

Aunque Java es un lenguaje Orientado a Objetos, **por dentro sus métodos funcionan de manera imperativa**. Cuando escribas el código dentro de una función en Java, estarás usando este paradigma: declararás variables, las modificarás y usarás bucles y condicionales para controlar el flujo.

3.3 Paradigma estructurado

El **paradigma estructurado** es una evolución directa del paradigma imperativo. Surgió a finales de los años 60 como respuesta a la crisis del software, provocada por la dificultad de mantener programas que crecían de forma desordenada y caótica (lo que se conocía como "código espagueti").

Su objetivo principal es mejorar la claridad, la calidad y el tiempo de desarrollo de un programa.

Fundamento Teórico: El Teorema de la Estructura

Este paradigma se basa en el **Teorema de la Estructura** (Böhm y Jacopini), que demuestra que cualquier algoritmo computable puede ser implementado utilizando únicamente tres estructuras de control lógicas . Esto elimina la necesidad de usar saltos incondicionales y caóticos (el famoso comando `GOTO`) que hacían el código ilegible.

Las tres estructuras fundamentales que definen este paradigma y que utilizarás en Java son:

1. **Secuencia:** Las instrucciones se ejecutan una tras otra en orden lineal. Es el bloque fundamental de construcción.
    - *En Java:* Simplemente escribir una línea de código debajo de otra.
2. **Selección (Estructuras condicionales):** Permite bifurcar el flujo del programa basándose en una condición lógica. Se decide qué bloque de código ejecutar.
    - *En Java:* Sentencias `if`, `else`, `switch`.
3. **Iteración (Estructuras de repetición):** Permite repetir un bloque de instrucciones mientras se cumpla una condición, evitando copiar y pegar código.
    - *En Java:* Bucles `while`, `do-while`, `for`.

Modularidad y Subrutinas

Además de las estructuras de control, el paradigma estructurado promueve la **división del problema** en partes más pequeñas y manejables (divide y vencerás).

- En lugar de escribir un programa gigante de 1000 líneas seguidas, se divide en **subrutinas** o funciones (que en Java llamaremos **métodos**).
- Cada subrutina realiza una tarea específica y puede ser llamada desde cualquier parte del programa.
- Esto facilita enormemente la detección de errores (depuración) y la reutilización de código.

3.4 Paradigma orientado a objetos (POO)

El **Paradigma Orientado a Objetos** (POO) representa un cambio radical en la forma de pensar el software respecto a la programación estructurada. Es el enfoque dominante en la industria actual y el eje central de este módulo profesional.

Concepto Fundamental

Mientras que el paradigma estructurado se centraba en la *lógica* (acciones, verbos, funciones) y trataba los datos como algo secundario que se movía de un lado a otro, la POO pone a los **datos** (sustantivos, cosas) en el centro.

La idea base es intentar modelar el software tal y como percibimos el mundo real. El mundo no está hecho de funciones flotantes, sino de **objetos** que interactúan entre sí.

- **Enfoque Estructurado:** "Tengo una lista de libros (datos) y una función `prestarLibro()` (lógica) que recibe esa lista".
- **Enfoque Orientado a Objetos:** "Tengo un objeto `Libro` que sabe prestarse a sí mismo (contiene sus propios datos y su lógica)".

Elementos Clave: Clases y Objetos

Aunque profundizaremos en la Unidad 2, es vital entender ahora la diferencia conceptual:

1. **Clase (El Molde):** Es una definición abstracta, una plantilla. Define qué propiedades y comportamientos tendrá un tipo de entidad.
    - *Ejemplo:* La clase `Coche` define que todos los coches tienen color, marca y pueden acelerar.
2. **Objeto (La Instancia):** Es la concreción de esa clase en la memoria. Es una entidad individual con valores propios.
    - *Ejemplo:* Un objeto específico (mi coche rojo) creado a partir del molde `Coche`.

Principios o Pilares Básicos

La POO se sustenta en cuatro pilares que permiten crear código reutilizable, modular y fácil de mantener:

1. **Encapsulamiento:** Protege la información del objeto. Los datos internos (atributos) se ocultan y solo se accede a ellos a través de métodos públicos. Esto evita modificaciones accidentales ("caja negra").
2. **Abstracción:** Nos permite centrarnos en "qué hace" un objeto sin preocuparnos por "cómo lo hace" internamente.
3. **Herencia:** Permite crear nuevas clases basadas en otras ya existentes, reutilizando su código y extendiendo sus funcionalidades. Es clave para establecer jerarquías (ej. `CocheDeportivo` hereda de `Coche`).
4. **Polimorfismo:** Capacidad de que objetos de diferentes clases respondan al mismo mensaje (método) de formas distintas.

Importante: Java es un lenguaje diseñado desde su nacimiento para ser **Orientado a Objetos**.

- En Java, **todo** (o casi todo) es un objeto.
- No puedes escribir funciones sueltas como en C; todo el código debe estar dentro de una **Clase**.
- Aprender Java es, en esencia, aprender a diseñar clases y a hacer que los objetos colaboren entre sí.

3.5 Paradigma funcional

El **Paradigma Funcional** es un enfoque de programación de tipo **declarativo** (se centra en el "qué" se quiere conseguir y no tanto en el "cómo" paso a paso).

A diferencia del paradigma imperativo, que cambia el estado de las variables constantemente, la programación funcional se basa en el concepto matemático de **función**: una relación entre una entrada y una salida donde no existen efectos secundarios.

Características Principales

Para entender este paradigma, debemos cambiar nuestra mentalidad de "dar órdenes" a "transformar datos":

1. **Inmutabilidad:** En la programación funcional pura, las variables no cambian de valor. Una vez que creas un dato, se mantiene así. Si quieres modificarlo, no lo sobrescribes; creas una copia nueva con el cambio aplicado. Esto elimina muchos errores difíciles de detectar.
2. **Funciones Puras:** Son funciones que, para una misma entrada, siempre devuelven la misma salida. No dependen de nada externo (como una variable global o la hora del sistema) ni modifican nada fuera de ellas.
   - *Ejemplo:* `suma(2, 3)` siempre devuelve `5`.
3. **Funciones de Orden Superior:** Las funciones son tratadas como "ciudadanos de primera clase". Esto significa que puedes pasar una función como argumento a otra función, o devolver una función como resultado.

Comparación: Imperativo vs. Funcional

Imagina que tienes una lista de números y quieres obtener el cuadrado de los pares.

- **Enfoque Imperativo (Tradicional):** Crearías una lista vacía, recorrerías la lista original con un bucle `for`, pondrías un `if` para ver si es par, calcularías el cuadrado y lo añadirías a la nueva lista.
- **Enfoque Funcional:** Tomarías la lista original y le aplicarías una cadena de transformaciones: `FiltraPares` → `CalculaCuadrados`. No te preocupas de los bucles internos.

Su relevancia en Java (Lambdas y Streams)

Aunque Java nació como un lenguaje puramente Orientado a Objetos, a partir de la versión **Java 8** introdujo características del paradigma funcional para modernizarse.

Durante el curso, verás estas pinceladas funcionales cuando utilices:

- **Expresiones Lambda:** Una forma concisa de escribir funciones anónimas (bloques de código que se pueden pasar como variables).
- **Streams (Flujos):** Una forma potente de procesar colecciones de datos (listas, arrays) de manera declarativa y funcional, muy útil para manipular grandes volúmenes de información.

3.6 Comparación entre paradigmas

Ahora que conocemos los distintos enfoques, es fundamental entender que no son enemigos, sino herramientas diferentes. Elegir el paradigma correcto depende de la naturaleza del problema a resolver.

A continuación, comparamos los tres grandes bloques que hemos visto, destacando cómo gestionan los elementos clave del software.

Tabla Comparativa General

Característica	Imperativo / Estructurado	Orientado a Objetos (POO)	Funcional
Elemento central	**La Instrucción / Algoritmo.** Se centra en los verbos ("calcular", "imprimir").	**El Objeto.** Se centra en los sustantivos ("Usuario", "Factura").	**La Función.** Se centra en la transformación matemática de valores.
Manejo de Datos	Los datos y las funciones están separados. Los datos son pasivos y se mueven de un lado a otro.	Datos y métodos están unidos (encapsulados). El objeto es responsable de su propia información.	Los datos son **inmutables**. No se modifican, se crean nuevos a partir de los anteriores.
Estado	El estado (valores de variables) cambia constantemente línea a línea.	El estado está protegido dentro del objeto y cambia mediante mensajes (métodos).	Se intenta evitar el estado mutable (stateless) para evitar efectos secundarios.
Reutilización	Baja/Media. Se reutilizan funciones o librerías.	**Alta.** Gracias a la herencia y el polimorfismo, es fácil extender código existente.	Alta. Las funciones puras son muy fáciles de combinar y reutilizar.
Ventaja principal	Es fácil de entender al principio. Control total sobre el hardware.	Ideal para **sistemas grandes y complejos**. Facilita el mantenimiento y el trabajo en equipo.	Ideal para procesamiento paralelo (concurrencia) y sistemas matemáticos/lógicos.

Diferencia clave: Estructurado vs. Orientado a Objetos

Dado que Java y este módulo se basan en la transición de lo estructurado a los objetos, esta es la distinción más crítica:

- **Enfoque Estructurado (C, Pascal, BASIC):** Piensas en una "receta". El programador es el chef que controla todo.
    - *Ejemplo:* `hacerSonar(miCoche)`
- **Enfoque Orientado a Objetos (Java, C#, Python):** Piensas en un "ecosistema". El programador crea objetos y deja que interactúen.

- *Ejemplo:* `miCoche.sonar()` → Le pides al objeto coche que ejecute su propio comportamiento.

4. Lenguajes de programación

4.1 Definición de lenguaje de programación

Un **lenguaje de programación** es un idioma formal diseñado para comunicar instrucciones a una máquina, particularmente a una computadora.

Del mismo modo que los humanos usamos el español o el inglés para comunicarnos entre nosotros, los programadores utilizan estos lenguajes para describir los algoritmos que la máquina debe ejecutar.

Componentes Fundamentales

A diferencia del lenguaje natural (que es flexible y a veces ambiguo), un lenguaje de programación es estricto y riguroso. Se compone de dos dimensiones inseparables:

1. **Sintaxis (La Forma):** Es el conjunto de reglas gramaticales que definen cómo se deben escribir las instrucciones correctamente. Dicta el orden de las palabras, el uso de paréntesis, puntos y comas, etc.

   - *Ejemplo en Java:* Es obligatorio terminar las sentencias con `;`. Si escribes `int x = 5` sin el punto y coma, hay un **error de sintaxis** y el programa no funciona.

2. **Semántica (El Significado):** Se refiere a la lógica y el significado de esas instrucciones. Una frase puede ser sintácticamente correcta pero no tener sentido (o no hacer lo que queríamos).

   - *Ejemplo:* `int resultado = 5 / 0;` está bien escrito (sintaxis correcta), pero semánticamente provoca un error grave (división por cero) que detiene el programa.

Función Principal

El lenguaje de programación actúa como un **intermediario o traductor**.

- **Nosotros** pensamos en conceptos abstractos (bucles, objetos, sumas).
- **El Hardware** solo entiende pulsos eléctricos (0 y 1).
- **El Lenguaje** nos permite escribir en términos casi humanos (como `if`, `while`, `class`) y se encarga (mediante otras herramientas) de que eso se convierta en ceros y unos.

4.2 Clasificación de lenguajes: bajo nivel vs alto nivel

Los lenguajes de programación se clasifican habitualmente según su **nivel de abstracción**. Es decir, cuánto se alejan de los detalles físicos de la máquina (ceros, unos, registros de memoria) y cuánto se acercan al lenguaje humano y a la lógica del problema.

A. Lenguajes de Bajo Nivel

Son lenguajes totalmente dependientes de la máquina. Están diseñados para controlar directamente el hardware. El programador debe conocer la arquitectura interna del procesador para usarlos.

1. **Lenguaje Máquina (Primera Generación):**

   - Es el único que el ordenador entiende directamente sin traducción.
   - Consiste exclusivamente en secuencias de números binarios (`01101001...`).
   - *Desventaja:* Es increíblemente difícil de escribir, leer y depurar para un humano. Un pequeño error en un bit puede colgar el sistema.

2. **Lenguaje Ensamblador (Segunda Generación):**

   - Utiliza nemotécnicos (palabras cortas como `ADD`, `MOV`, `SUB`) para representar las instrucciones básicas del procesador en lugar de números.
   - *Ejemplo:* `MOV AL, 61h` (Mover el valor 61 hexadecimal al registro AL).
   - *Uso actual:* Se sigue usando para programación de sistemas críticos (drivers, kernels de sistemas operativos) o microcontroladores donde la velocidad y el control del hardware son vitales.

B. Lenguajes de Alto Nivel

Son lenguajes diseñados para ser entendidos por humanos. Se abstraen (se olvidan) de los detalles del hardware. Java, C++, Python y la inmensa mayoría de lenguajes modernos pertenecen a esta categoría.

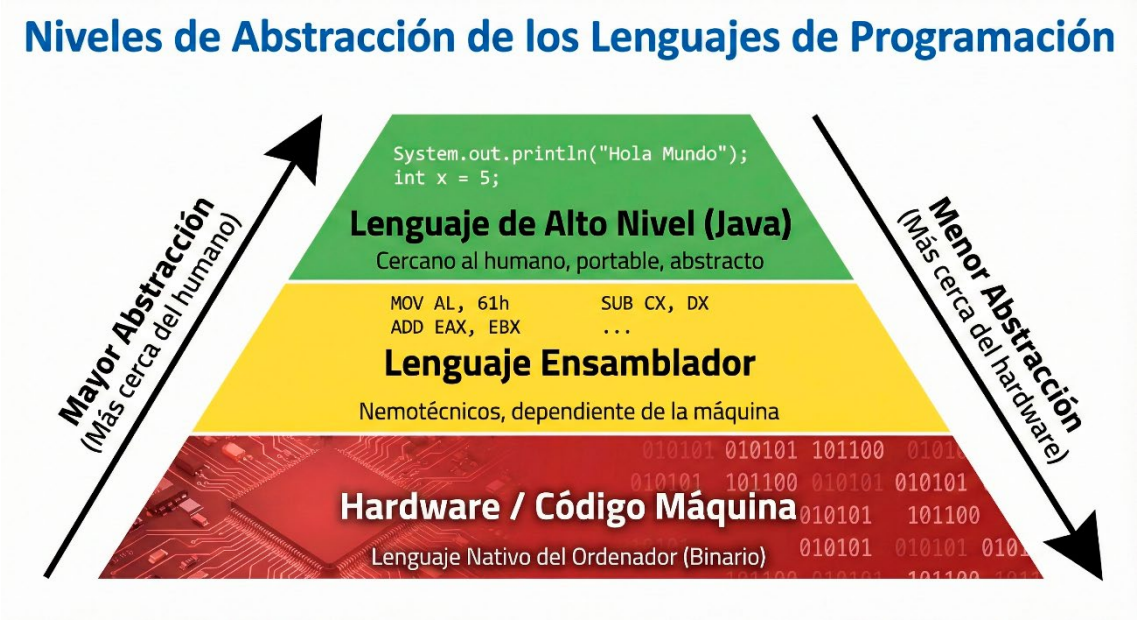
- **Características:**

  - **Portabilidad:** Un programa escrito en alto nivel puede funcionar en diferentes ordenadores sin necesidad de reescribirlo (siempre que exista un traductor/compilador para esa máquina).
  - **Legibilidad:** Usan palabras en inglés (`if`, `while`, `class`, `print`) y notación matemática estándar (`a = b + c`).
  - **Productividad:** Permiten escribir programas complejos con muchas menos líneas de código y en menos tiempo.

Comparativa Visual

Característica	Bajo Nivel (Ensamblador)	Alto Nivel (Java)
Cercanía	Cerca del Hardware (CPU)	Cerca del Humano (Lógica)
Entendimiento	Difícil para humanos	Fácil para humanos
Portabilidad	No portable (específico de la CPU)	Muy portable ("Write once, run anywhere")
Control	Control total de memoria y registros	El lenguaje gestiona la memoria por ti

Característica	Bajo Nivel (Ensamblador)	Alto Nivel (Java)
Velocidad	Máxima (si se optimiza bien)	Ligeramente menor (por la traducción)



Más info: Java es un lenguaje de **Alto Nivel**. Cuando escribas en Java, no tendrás que preocuparte por en qué registro del procesador se guarda una variable; el entorno de Java se encargará de gestionar esos detalles de bajo nivel por ti.

4.3 Lenguajes compilados vs interpretados

Como vimos en el punto anterior, los ordenadores no entienden el código de alto nivel (como el código fuente que tú escribes). Necesitan una **traducción** a código máquina (binario). Dependiendo de *cuándo* y *cómo* se realice esta traducción, los lenguajes se dividen en dos grandes grupos.

A. Lenguajes Compilados

En estos lenguajes, la traducción completa del programa se realiza **antes** de su ejecución.

1. **Proceso:** El programador utiliza un software llamado **compilador** que lee todo el código fuente de una vez, comprueba errores y lo traduce a un fichero ejecutable (código máquina nativo para ese sistema operativo, como un `.exe` en Windows).
2. **Ejecución:** Una vez compilado, el programa se puede ejecutar tantas veces como se quiera sin necesitar el código fuente ni el compilador.
3. **Ventajas:** Son muy rápidos y eficientes en la ejecución, ya que el trabajo de traducción ya está hecho.
4. **Desventajas:** Cada vez que haces un pequeño cambio en el código, debes volver a compilar todo el programa. Además, el ejecutable solo sirve para el sistema operativo donde se compiló (no es portable por defecto).

- *Ejemplos:* C, C++, Go, Rust.

B. Lenguajes Interpretados

En estos lenguajes, la traducción se realiza **durante** la ejecución.

1. **Proceso:** No existe un compilador que genere un ejecutable. En su lugar, hay un software llamado **intérprete** que lee el código fuente línea por línea, lo traduce a código máquina en ese instante y lo ejecuta inmediatamente.
2. **Ejecución:** Necesitas tener el intérprete instalado en el ordenador para correr el programa.
3. **Ventajas:** Son muy flexibles y portables (el código fuente funciona en cualquier sitio donde esté el intérprete). El ciclo de desarrollo es rápido (escribir y probar al instante).
4. **Desventajas:** Son más lentos en ejecución, ya que el ordenador debe "perder tiempo" traduciendo cada línea cada vez que se ejecuta el programa.

    - *Ejemplos:* Python, JavaScript, PHP.

C. El enfoque híbrido: El caso de Java

Es fundamental que entiendas que **Java** (el lenguaje de este módulo) es especial porque utiliza un sistema **mixto**.

- **No es 100% compilado:** No genera código máquina directo para Windows o Linux. Compila a un código intermedio llamado **Bytecode**.
- **No es 100% interpretado:** No lee tu código fuente original al ejecutar, sino que interpreta ese Bytecode optimizado.

  En resumen: Java combina lo mejor de los dos mundos.

  - Se **compila** para detectar errores y generar un formato eficiente (`.class`).
  - Se **interpreta** (por la Máquina Virtual) para que ese mismo archivo funcione en cualquier ordenador del mundo.

4.4 Traducción del código: compilación, interpretación y máquina virtual

Como hemos visto, el ordenador no entiende tu código fuente. Para que se ejecute, debe ocurrir un proceso de **traducción**. En el ecosistema de Java, este proceso es único porque combina tres elementos clave: el compilador, el bytecode y la máquina virtual.

1. El Proceso de Compilación (De .java a .class)

En Java, el primer paso es la compilación, pero con una diferencia importante respecto a lenguajes tradicionales como C++.

- **¿Qué ocurre?** El compilador de Java (`javac`) lee tu archivo de código fuente (con extensión `.java`).
- **¿Qué genera?** No genera código máquina para tu CPU (que sería un `.exe`). En su lugar, genera un archivo con extensión `.class` que contiene **Bytecode**.

- **¿Qué es el Bytecode?** Es un código intermedio, una especie de "lenguaje máquina universal" que no está diseñado para ningún procesador físico real, sino para una procesadora abstracta (virtual).

  Resultado: Tienes un archivo `.class` optimizado y verificado, pero que tu Windows o Mac todavía no saben ejecutar directamente.

2. La Máquina Virtual de Java (JVM)

Aquí entra la pieza maestra de la arquitectura Java. La **Java Virtual Machine (JVM)** es un programa que simula ser un ordenador dentro de tu ordenador.

- **Función:** Su trabajo es actuar de intérprete. Lee el Bytecode (`.class`) y lo traduce en tiempo real al código máquina nativo específico del ordenador donde se está ejecutando (sea Windows, Linux, Android, etc.).
- **Abstracción:** Gracias a la JVM, tú como programador no necesitas preocuparte por si el usuario tiene un procesador Intel o AMD, o si usa Windows 10 o Ubuntu. La JVM se encarga de lidiar con el hardware.

3. Interpretación y Ejecución (JIT)

Antiguamente, la interpretación era lenta. Sin embargo, las JVM modernas utilizan una técnica llamada **Compilación Just-In-Time (JIT)**.

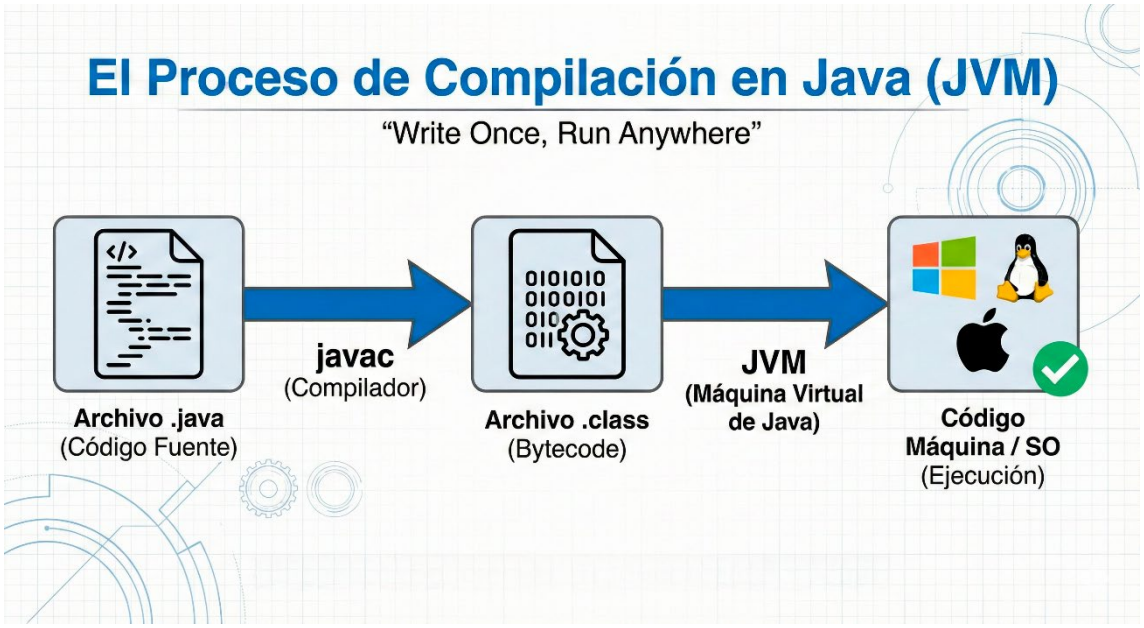
- El compilador JIT, que vive dentro de la JVM, detecta qué partes del código se usan mucho (puntos calientes) y las compila a código máquina nativo en ese mismo instante para que vayan a máxima velocidad.
- Esto permite que Java tenga la portabilidad de un lenguaje interpretado con una velocidad muy cercana a uno compilado.

Resumen del Flujo en Java

Para que te quede claro el ciclo de vida de tu código en este módulo:

1. **Escritura:** Creas `HolaMundo.java` (Código fuente legible).
2. **Compilación:** Ejecutas `javac` y obtienes `HolaMundo.class` (Bytecode).
3. **Ejecución:** La JVM lee ese archivo y "habla" con tu sistema operativo para mostrar el resultado en pantalla.

   El lema de Java: *"Write Once, Run Anywhere"* (Escribe una vez, ejecuta en cualquier lugar). Solo necesitas compilar una vez; ese mismo archivo `.class` funcionará en cualquier máquina que tenga instalada una JVM.



4.5 Características generales de los lenguajes modernos

Los lenguajes de programación han evolucionado drásticamente desde los años 50. Hoy en día, lenguajes modernos como **Java**, C#, Python o Swift comparten una serie de características diseñadas para aumentar la productividad del programador y la fiabilidad del software.

A continuación, detallamos las características que definen el entorno en el que vas a trabajar durante este curso.

1. Abstracción y Legibilidad

La prioridad actual es que el código sea comprensible para los humanos, no solo para las máquinas.

- **Sintaxis expresiva:** Se utilizan palabras clave claras y estructuras lógicas que imitan el lenguaje natural.
- **Ocultación de complejidad:** El programador no necesita saber cómo funciona el disco duro para guardar un archivo; utiliza una instrucción simple y el lenguaje se encarga de los detalles técnicos "sucios".

2. Gestión Automática de Memoria (Garbage Collection)

Esta es, quizás, la característica que más facilita la vida al programador en lenguajes como Java.

- **En el pasado (C/C++):** El programador tenía que pedir memoria manualmente para crear un objeto y, lo más importante, tenía que acordarse de **liberarla** cuando dejaba de usarla. Si olvidaba hacerlo, el programa consumía toda la RAM y bloqueaba el ordenador ("memory leak").
- **En lenguajes modernos (Java):** Existe un proceso automático llamado **Recolector de Basura (Garbage Collector)**. Este sistema vigila constantemente la memoria, detecta qué objetos ya no se están utilizando y los elimina automáticamente para liberar recursos.

3. Tipado Fuerte y Seguridad

Para evitar errores catastróficos, los lenguajes modernos suelen incorporar sistemas de tipos robustos.

- **Seguridad de tipos:** El lenguaje impide que realices operaciones absurdas, como sumar "Peras" con "3.14". Esto detecta muchos errores antes incluso de que ejecutes el programa.
- **Manejo de Excepciones:** Proveen mecanismos estructurados (`try-catch`) para controlar errores inesperados (como que se caiga internet) sin que el programa se cierre abruptamente.

4. Bibliotecas Estándar (API) Ricas

Los lenguajes modernos vienen con "baterías incluidas". No necesitas programar desde cero cosas básicas.

- Ofrecen inmensas colecciones de código prefabricado y probado (Librerías o API) para realizar tareas comunes: conectarse a internet, dibujar ventanas, manejar fechas, leer bases de datos, etc.
- *En el curso:* Aprenderás a usar la **API de Java** para no "reinventar la rueda".

5. Orientación a Objetos y Multiparadigma

Como vimos anteriormente, la estructura base es la **Orientación a Objetos** para organizar el código de forma modular y reutilizable. Sin embargo, los lenguajes modernos son flexibles e incorporan características funcionales (como las expresiones Lambda en Java) para simplificar tareas específicas.

6. Independencia de la Plataforma (Portabilidad)

La capacidad de ejecutar el mismo código en diferentes sistemas operativos (Windows, Linux, macOS, Android) sin necesidad de reescribirlo. En el caso de Java, esto se logra mediante la Máquina Virtual.

5. Herramientas y entornos para el desarrollo de programas

5.1 Editores de texto para programación

La herramienta más básica e imprescindible para cualquier programador es el editor de texto. Es el lugar donde escribimos el **código fuente** de nuestros programas.

¿Qué es un editor de texto para programación?

Es un software diseñado específicamente para editar archivos de **texto plano** (sin formato). A diferencia de un procesador de textos convencional (como Microsoft Word), un editor de código no añade estilos ocultos, negritas, ni tipos de letra al archivo; solo guarda los caracteres puros que forman las instrucciones.

Importante: Nunca intentes programar en Word. Los procesadores de texto añaden caracteres ocultos de formato que el compilador de Java no entenderá, provocando errores inmediatos.

Características esenciales

Aunque podrías escribir código con el "Bloc de Notas" de Windows, sería muy incómodo y propenso a errores. Los editores modernos para programación incluyen ayudas visuales:

1. **Resaltado de sintaxis (Syntax Highlighting):** Colorea automáticamente el código según su función. Por ejemplo, en Java, las palabras reservadas como class o public pueden aparecer en azul, los textos entre comillas en verde y los números en rojo. Esto facilita enormemente la lectura y ayuda a detectar errores tipográficos al instante.
2. **Numeración de líneas:** Fundamental para la depuración. Cuando el compilador te diga "Error en la línea 45", necesitas ver rápidamente dónde está esa línea.
3. **Autocompletado básico:** Sugiere cerrar paréntesis, corchetes o comillas automáticamente para evitar que se te olviden.
4. **Indentación automática:** Ayuda a mantener el código ordenado, tabulando las líneas automáticamente cuando abres una nueva estructura (como un if o un bucle).

Ejemplos comunes

Algunos editores ligeros muy populares antes de dar el salto a entornos más complejos son:

- **Notepad++:** Un clásico en Windows, muy ligero y rápido.
- **Sublime Text:** Visualmente muy limpio y potente.
- **Visual Studio Code (VS Code):** Aunque técnicamente es un editor, es tan potente que roza la categoría de IDE (que veremos en el siguiente punto). Es uno de los más usados actualmente.

5.2 Entornos de desarrollo integrados (IDE)

Si un editor de texto es como tener un destornillador y un martillo sueltos, un **IDE (Integrated Development Environment)** es como tener un taller mecánico completo y automatizado.

¿Qué es un IDE?

Un Entorno de Desarrollo Integrado es una aplicación informática que proporciona servicios integrales para facilitar al programador el desarrollo de software. Su objetivo es unificar todas las herramientas necesarias en una sola interfaz gráfica, evitando que tengas que saltar entre diferentes programas (editor, consola, compilador, etc.).

Componentes Principales

Un IDE no es una herramienta única, sino la suma de varias herramientas conectadas entre sí:

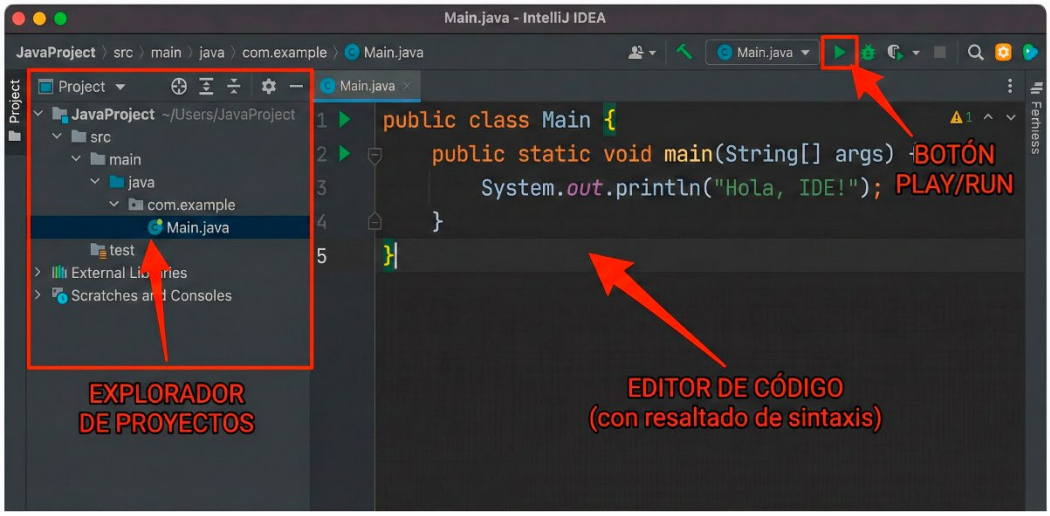
1. **Editor de código inteligente:** Mucho más avanzado que los editores simples. Incluye autocompletado (IntelliSense), que sugiere nombres de métodos y variables mientras escribes, y detección de errores en tiempo real (te subraya en rojo el código incorrecto antes incluso de compilar).

2. **Compilador/Intérprete integrado:** Permite traducir y ejecutar el programa con un solo clic (normalmente un botón "Play" o "Run"). El IDE se encarga de llamar a los comandos internos (javac, java) por ti.
3. **Depurador (Debugger):** Herramienta visual para ejecutar el programa paso a paso y ver qué está ocurriendo dentro de las variables en cada momento (veremos esto en el punto 5.4).
4. **Gestión de proyectos:** Permite organizar cientos de archivos de código, imágenes y librerías en una estructura de carpetas lógica, facilitando la navegación por proyectos complejos.

Ventajas en el aprendizaje de Java

El uso de un IDE es obligatorio según los criterios de evaluación del módulo, ya que permite:

- **Productividad:** Escribes código mucho más rápido gracias a la generación automática de estructuras (el IDE puede escribir por ti los constructores, los `getters` y `setters`, etc.).
- **Aprendizaje:** Al marcarte los errores al instante y sugerirte correcciones, aprendes la sintaxis del lenguaje más rápido que si tuvieras que compilar manualmente y leer errores crípticos en la consola.



Ejemplos de IDEs para Java

En el ámbito profesional y educativo, los tres grandes referentes son:

- **Eclipse:** Históricamente el más usado en educación. Es gratuito, de código abierto y muy robusto, aunque su interfaz puede parecer algo antigua.
- **NetBeans:** Muy común en entornos educativos por su simplicidad y buena integración inicial con Java.
- **IntelliJ IDEA:** El estándar actual en la industria. Es muy potente e inteligente, aunque consume más recursos del ordenador.

5.3 Compiladores e intérpretes

En el desarrollo de software, el código fuente que escribimos en el editor es solo texto. Para que cobre vida, necesitamos herramientas específicas que realicen la "traducción". En el entorno de Java, utilizamos dos programas distintos que trabajan en cadena.

A. El Compilador (`javac`)

Es la herramienta encargada de la primera fase de traducción. En el Kit de Desarrollo de Java (JDK), este programa se llama `javac`.

- **Función:** Lee los archivos `.java` (código fuente), verifica que la sintaxis sea perfecta y los transforma en archivos `.class` (bytecode).
- **Comportamiento:** Es una herramienta estática. No ejecuta el programa; solo lo analiza.
- **Detección de errores:** Si el compilador encuentra un fallo (como un punto y coma faltante o una variable mal escrita), detiene el proceso y muestra una lista de "Errores de compilación". Si hay errores aquí, no se genera el archivo `.class` y, por tanto, no se puede ejecutar nada.
- **En el IDE:** Normalmente el IDE compila automáticamente cada vez que guardas el archivo (por eso ves las líneas rojas de error aparecer y desaparecer casi al instante).

B. El Intérprete (`java`)

Es la herramienta encargada de la ejecución. Es la puerta de entrada a la Máquina Virtual de Java (JVM). En el JDK, este comando es simplemente `java`.

- **Función:** Toma los archivos `.class` generados por el compilador, carga el bytecode en la memoria y hace que el procesador ejecute las instrucciones paso a paso.
- **Comportamiento:** Es una herramienta dinámica. Funciona en "tiempo de ejecución".
- **Detección de errores:** Si ocurre algo inesperado mientras el programa funciona (como intentar dividir por cero o leer un archivo que no existe), el intérprete lanza una "Excepción" (error de ejecución) y detiene el programa.

Diferencias prácticas para el programador

Herramienta	Cuándo actúa	Qué detecta	Resultado exitoso
Compilador	Antes de ejecutar (Tiempo de Compilación)	Errores de sintaxis y tipos	Un archivo binario (`.class`)
Intérprete	Durante la ejecución (Tiempo de Ejecución)	Errores lógicos y excepciones	El resultado del programa en pantalla

Más info: Aunque en el IDE solo pulses un botón verde de "Play", internamente el IDE está llamando primero al **Compilador** y, si este da el visto bueno, llama inmediatamente al **Intérprete**. Son dos procesos separados pero automatizados por el entorno.

5.4 Depuradores (Debuggers)

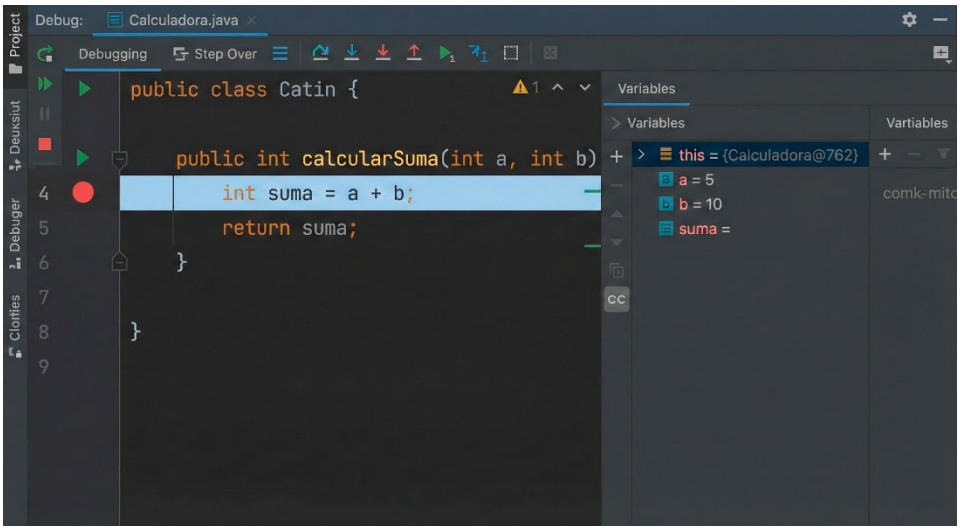
Escribir código es solo la mitad del trabajo; la otra mitad es averiguar por qué no funciona como esperabas. El **depurador** es una herramienta integrada en los entornos de desarrollo (IDE) que permite ejecutar el programa de manera controlada y detenerlo en cualquier punto para examinar qué está ocurriendo en su interior[1].

El término "debugging" (depuración) proviene de la anécdota histórica de eliminar una polilla real (*bug*) que causaba un fallo en un ordenador antiguo. Hoy en día, depurar significa eliminar errores de **lógica**.

¿Para qué sirve un depurador?

A diferencia del compilador, que te avisa de errores de sintaxis (como escribir mal una palabra), el depurador te ayuda a encontrar **errores lógicos** (el programa funciona, pero calcula mal una suma o entra en un bucle infinito).

El depurador te permite "congelar el tiempo" dentro de tu programa para ver el estado de la memoria en ese instante preciso.



Funcionalidades Clave

Para cumplir con los criterios de evaluación que exigen "probar y depurar los programas"[3], debes dominar cuatro acciones básicas en tu IDE (Eclipse, IntelliJ o NetBeans):

1. **Puntos de Ruptura (Breakpoints):** Es una marca (generalmente un punto rojo) que colocas en el margen de una línea de código. Cuando ejecutas el programa en "Modo Debug", la ejecución se detendrá automáticamente justo antes de ejecutar esa línea. Es como poner un semáforo en rojo para inspeccionar el tráfico.
2. **Ejecución paso a paso (Step by Step):** Una vez detenido el programa, puedes avanzarlo línea por línea manualmente.

    - **Step Over (Pasar por encima):** Ejecuta la línea actual y pasa a la siguiente.
    - **Step Into (Entrar):** Si la línea actual llama a un método, el depurador "entra" en ese método para ver cómo funciona por dentro.

3. **Inspección de Variables (Watches):** Te permite ver el valor exacto que tienen las variables en ese momento de la ejecución. Puedes ver cómo una variable contador pasa de 1 a 2, luego a 3, etc., y detectar en qué momento toma un valor incorrecto.
4. **Pila de Llamadas (Call Stack):** Te muestra el "rastro" de la ejecución: qué método llamó al método actual. Es vital para entender el flujo en programas complejos con muchas clases interactuando.

Diferencia entre Ejecutar (Run) y Depurar (Debug)

En tu IDE verás dos botones principales:

- **Run (Play verde):** Lanza el programa a velocidad normal. Úsalo cuando crees que todo funciona bien.
- **Debug (Bicho/Insecto):** Lanza el programa conectado al depurador. Úsalo cuando necesitas investigar un fallo.

Sabias qué: Un error común de principiante es intentar encontrar fallos leyendo el código una y otra vez o poniendo muchos `System.out.println` ("llegué aquí"). Usar el depurador es mucho más profesional, rápido y limpio.

5.5 Herramientas de análisis y diagnóstico

Más allá de que el programa compile (sin errores de sintaxis) y funcione (sin errores lógicos graves), buscamos que el código sea de **alta calidad**. Para ello, los entornos de desarrollo integran herramientas que analizan la "salud" del proyecto.

Podemos dividir estas herramientas en dos grandes categorías según cuándo actúan:

A. Análisis Estático (Sin ejecutar el código)

Son herramientas que leen tu código fuente como si fueran un corrector de estilo gramatical. Buscan patrones que, aunque son legales para el compilador, se consideran "malas prácticas" o posibles fuentes de errores futuros.

1. **Linters y Formateadores:**

- Se aseguran de que el código cumpla con las normas de estilo (por ejemplo, dónde poner las llaves `{`, cómo nombrar las variables).
- Ayudan a cumplir con el criterio de evaluación de "legibilidad".
- *Ejemplo en Java:* **Checkstyle** o **PMD**. Si llamas a una variable `x`, la herramienta te avisará: *"Nombre de variable poco descriptivo. Usa nombres completos"*.

2. **Detección de "Code Smells" (Malos olores):**

- Identifican trozos de código que son sintácticamente correctos, pero estructuralmente deficientes.
- *Ejemplos:* Métodos demasiado largos, demasiados `if/else` anidados, o código duplicado (copiar y pegar).
- *Herramienta típica:* **SonarLint** (se instala como plugin en el IDE y te subraya en amarillo las mejoras sugeridas).

B. Análisis Dinámico y Diagnóstico (Ejecutando el código)

Estas herramientas, conocidas como **Profilers** (Perfiladores), monitorizan el programa *mientras* está funcionando para ver cómo consume los recursos del ordenador.

1. **Análisis de Rendimiento (CPU Profiling):**

- Te dice cuánto tiempo pasa el procesador en cada método.
- Sirve para detectar "cuellos de botella": partes del código que hacen que la aplicación vaya lenta.

2. **Análisis de Memoria (Memory Profiling):**

- Visualiza cuánta memoria RAM está usando el programa y qué objetos están ocupando más espacio.
- Es vital para detectar **fugas de memoria** (Memory Leaks), un problema donde el programa olvida liberar memoria hasta que el ordenador se bloquea. Esto se relaciona directamente con el contenido de "Destrucción de objetos y liberación de memoria".

C. Refactorización (Refactoring)

Es una capacidad diagnóstica integrada en los IDEs modernos que permite **mejorar la estructura interna del código sin cambiar su comportamiento externo**.

- Si te das cuenta de que una clase es demasiado grande, el IDE te ofrece herramientas automatizadas para extraer parte de ese código y moverlo a una clase nueva de forma segura, actualizando todas las referencias automáticamente.
- Esto es esencial para mantener la **modularidad** y **mantenibilidad**.

5.6 Control de versiones

El **control de versiones** es un sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de modo que puedas recuperar versiones específicas más adelante.

Imagina que estás escribiendo un trabajo y guardas archivos como `trabajo_final.doc`, `trabajo_final_v2.doc`, `trabajo_final_definitivo.doc`. El control de versiones hace esto de forma profesional, automática y mucho más potente.

¿Por qué es fundamental?

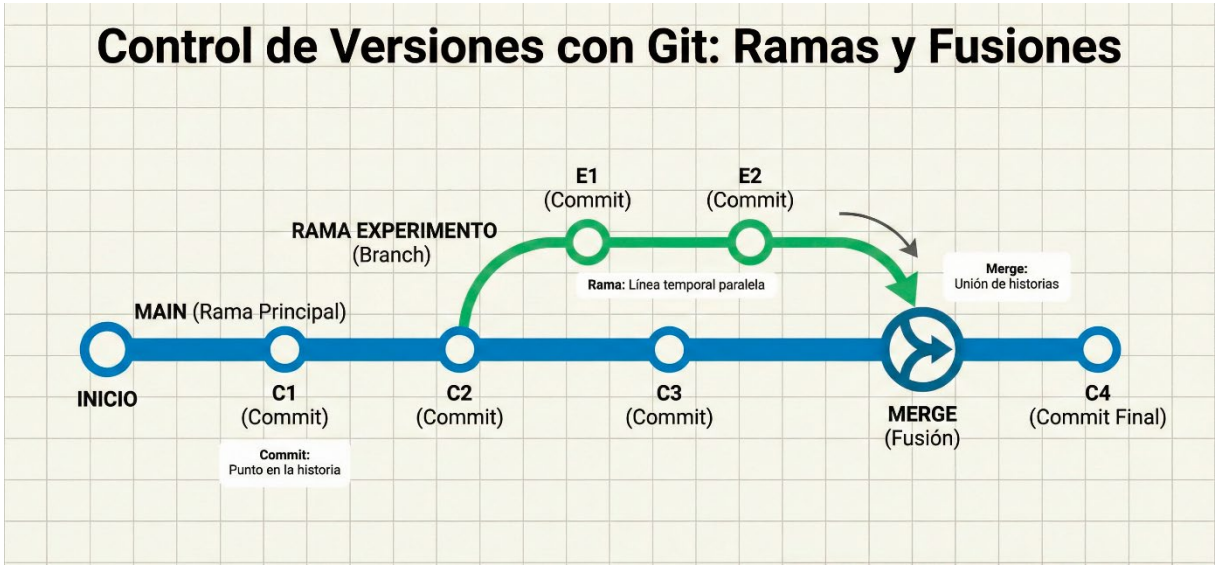
En el desarrollo de software, el código cambia constantemente. Sin un control de versiones:

1. **Miedo a romper nada:** Si modificas un código que funcionaba para intentar mejorarlo y lo rompes, no tienes una forma fácil de "deshacer" (Ctrl+Z tiene un límite).
2. **Trabajo en equipo imposible:** Si dos personas modifican el mismo archivo a la vez, el último en guardar sobrescribe el trabajo del primero.

Conceptos Clave

Aunque existen varios sistemas, el estándar mundial actual es **Git**. Debes familiarizarte con su vocabulario básico:

- **Repositorio (Repo):** Es la base de datos donde se almacenan todos los archivos de tu proyecto y el historial completo de cambios. Puede ser **local** (en tu ordenador) o **remoto** (en la nube, como GitHub o GitLab).
- **Commit (Confirmación):** Es la unidad básica de cambio. Es como sacar una "foto" del estado actual de tu proyecto. Cada commit tiene un identificador único y un mensaje explicativo (ej: "Areglado el fallo en la suma").
- **Rama (Branch):** Permite crear una línea temporal paralela. Puedes crear una rama "experimento" para probar algo nuevo sin afectar a la rama principal ("main" o "master"). Si el experimento sale bien, lo fusionas (**Merge**); si sale mal, lo borras sin romper nada.



Tipos de Control de Versiones

1. **Centralizado (ej. SVN):** Hay un único servidor que tiene todos los archivos. Si se cae el servidor, nadie puede trabajar ni guardar cambios.
2. **Distribuido (ej. Git):** Cada programador tiene una copia completa del historial en su ordenador. Es mucho más seguro y rápido. Este es el que se usa en la inmensa mayoría de empresas actuales.

Integración con el IDE

Como vimos en el punto 5.2, los IDEs modernos (IntelliJ, Eclipse, NetBeans) traen **Git integrado**.

- No necesitas escribir comandos en una pantalla negra (consola).
- Desde el propio editor puedes ver visualmente qué líneas has cambiado respecto a la versión anterior (aparecen marcadas en color en el margen).
- Puedes hacer "commits" y subir cambios a la nube con botones gráficos.

Importante: Utilizar un control de versiones te da la seguridad de poder experimentar con tu código Java sin miedo a perder el trabajo previo.

5.7 Gestión del proyecto y automatización (build)

Cuando empiezas a programar, sueles tener uno o dos archivos `.java`. Compilarlos pulsando un botón en el IDE es sencillo. Pero en el mundo real, un proyecto profesional puede tener cientos de archivos de código, decenas de librerías externas (bibliotecas), recursos gráficos y configuraciones. Gestionar todo esto manualmente es imposible.

Aquí entran en juego las **herramientas de construcción** (Build Tools) y gestión de proyectos.

¿Qué es la automatización de la construcción (Build)?

Es el proceso de automatizar la creación de un programa ejecutable a partir del código fuente. En lugar de que tú tengas que recordar qué archivos compilar y en qué orden, una herramienta sigue una "receta" predefinida para hacerlo por ti.

Un sistema de "build" se encarga automáticamente de tareas como:

1. **Compilar** todo el código fuente.
2. **Empaquetar** el resultado en un formato distribuible (en Java, por ejemplo, generar un archivo `.jar`).
3. **Ejecutar pruebas** automáticas para asegurar que nada se ha roto.
4. **Limpiar** archivos temporales antiguos antes de empezar.

Gestión de Dependencias (Librerías)

Esta es quizás la función más crítica para el desarrollador moderno.

Como verás en el curso, a menudo necesitarás funcionalidades que ya existen (leer un PDF, conectar a una base de datos, crear gráficos). En lugar de escribir ese código desde cero, utilizas librerías externas.

- **Método manual (Antiguo):** Buscas la librería en internet, descargas el archivo `.jar`, lo copias a tu proyecto y configuras el IDE para que lo reconozca. Si la librería se actualiza, tienes que repetir todo.
- **Método automatizado (Moderno):** En un archivo de configuración (como `pom.xml` en Maven), simplemente escribes: *"Necesito la librería PDFBox, versión 2.0"*. La herramienta de construcción se conecta a internet, la descarga automáticamente y la configura por ti.

Herramientas Estándar en Java

En el ecosistema Java, existen dos grandes gigantes que probablemente encuentres integrados en tu IDE:

1. **Maven:**

   - Es el estándar más extendido.
   - Usa un archivo llamado `pom.xml` para definir el proyecto.
   - Prioriza la "convención sobre configuración" (si sigues sus normas de carpetas, funciona casi solo).

2. **Gradle:**

- Más moderno y flexible (usado oficialmente por Google para Android).
- Es más rápido en proyectos gigantes, pero tiene una curva de aprendizaje un poco más alta.

6. Errores y calidad de los programas

6.1 Tipos de errores

Para poder corregir un fallo, primero debemos entender su naturaleza. No todos los errores son iguales ni aparecen en el mismo momento. Según la fase en la que se detectan y su causa, los clasificamos en tres grandes categorías.

A. Errores de sintaxis (Compile-time errors)

Son los errores más básicos y fáciles de corregir. Ocurren cuando el código escrito no sigue las reglas gramaticales del lenguaje de programación.

- **¿Cuándo aparecen?** En tiempo de compilación. El compilador (o el IDE) los detecta antes de que intentes ejecutar el programa.
- **Consecuencia:** El programa **no se puede ejecutar**. No se genera el archivo `.class` (bytecode).
- **Causas típicas en Java:**
    - Olvidar un punto y coma `;` al final de una sentencia.
    - Escribir mal una palabra reservada (`publik` en lugar de `public`).
    - No cerrar un paréntesis `(` o una llave `{`.
- **Detección:** El IDE te los marca inmediatamente con una línea roja ondulada bajo el código erróneo.

  Ejemplo: Escribir `System.out.println("Hola"` (falta cerrar el paréntesis y el punto y coma).

B. Errores de ejecución (Runtime errors o Excepciones)

Son errores que ocurren mientras el programa está funcionando, aunque la sintaxis sea correcta. El código está bien escrito gramaticalmente, pero intenta realizar una operación imposible o ilegal.

- **¿Cuándo aparecen?** En tiempo de ejecución (mientras usas el programa).
- **Consecuencia:** El programa se detiene abruptamente (se "cuelga" o "crashea") y suele mostrar un mensaje de error técnico en la consola (una Excepción).
- **Causas típicas:**
    - Intentar dividir un número por cero.
    - Intentar leer un archivo que no existe en el disco.
    - Intentar acceder a la posición 10 de una lista que solo tiene 5 elementos.

- **Manejo:** En Java, estos errores se gestionan mediante el control de excepciones (`try-catch`), un tema que se verá en profundidad en unidades posteriores.

  Ejemplo: El usuario introduce texto cuando el programa esperaba un número, y el programa intenta hacer una operación matemática con ese texto.

C. Errores lógicos (Bugs lógicos)

Son los errores más peligrosos y difíciles de detectar. El programa compila perfectamente y no se cuelga durante la ejecución, pero **no hace lo que debería hacer**.

- **¿Cuándo aparecen?** En tiempo de ejecución, pero no lanzan ningún aviso de error. Solo se notan porque el resultado es incorrecto.
- **Consecuencia:** El programa da datos falsos, guarda mal la información o se comporta de forma extraña.
- **Causa:** El fallo está en el **diseño del algoritmo** o en el planteamiento del programador, no en el código en sí.
- **Detección:** Son los que requieren el uso del **Depurador** (visto en el punto 5.4) y pruebas exhaustivas, ya que la máquina no puede avisarte de ellos (ella solo sigue tus instrucciones, aunque sean absurdas).

  Ejemplo: Quieres calcular el promedio de dos notas y programas `nota1 + nota2 / 2`.

  - El ordenador lo ejecutará felizmente, pero el resultado estará mal porque dividirá solo la `nota2` antes de sumar (por la precedencia de operadores).
  - La lógica correcta hubiera sido `(nota1 + nota2) / 2`.

Resumen visual de los tipos:

Tipo de Error	¿Compila?	¿Ejecuta?	¿Quién avisa?	Dificultad
Sintaxis	NO	NO	El Compilador/IDE	Baja
Ejecución	SÍ	Hasta el fallo	La Máquina Virtual (JVM)	Media
Lógico	SÍ	SÍ	Nadie (El usuario o tester)	Alta

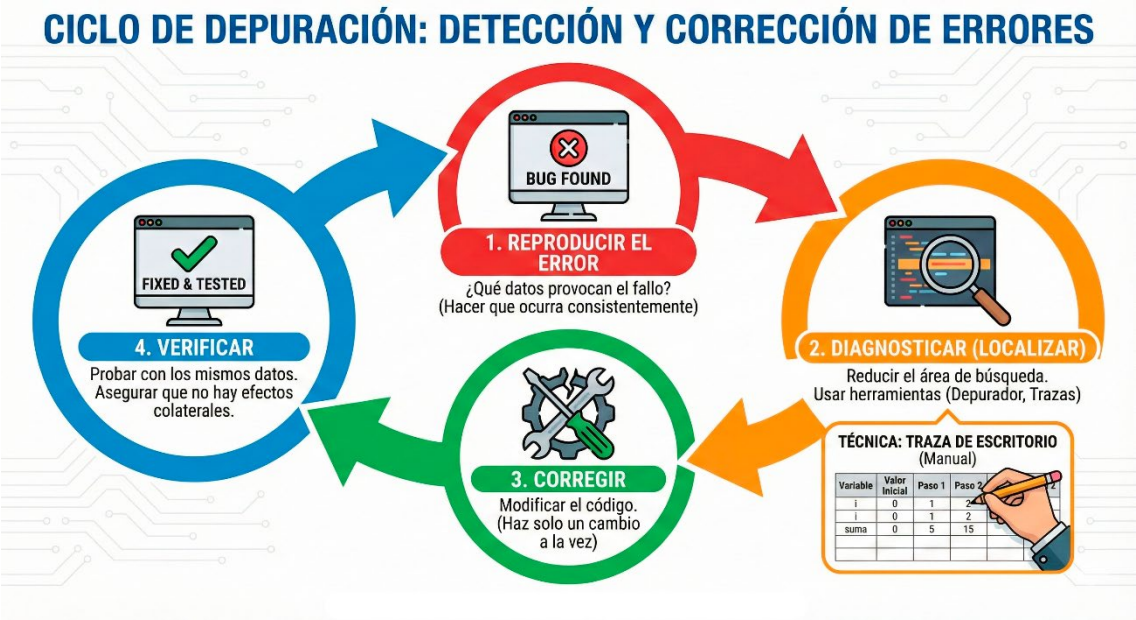
6.2 Detección y corrección de errores

La depuración (o *debugging*) no es un acto de adivinanza; es un proceso sistemático similar a una investigación policial o médica. Requiere paciencia, observación y seguir unos pasos lógicos para aislar la causa del problema.

A. El Ciclo de Depuración

Para corregir un error (especialmente los lógicos o de ejecución), se recomienda seguir este ciclo iterativo:

1. **Reproducir el error:** Antes de intentar arreglar nada, debes ser capaz de hacer que el fallo ocurra de forma consistente. Si no puedes reproducirlo, no podrás saber si lo has arreglado.
    - *Pregunta:* "¿Qué datos de entrada provocan el fallo?"
2. **Diagnosticar (Localizar):** Es la fase más difícil. Debes reducir el área de búsqueda hasta encontrar la línea exacta o el bloque de código culpable.
    - *Estrategia:* "Divide y vencerás". Comenta la mitad del código; si el error persiste, está en la mitad activa. Si no, está en la comentada.
3. **Corregir:** Modificar el código para solucionar el problema.
    - *Consejo:* Haz solo un cambio a la vez. Si cambias cinco cosas de golpe y funciona, no sabrás cuál fue la solución (o si has introducido nuevos errores).
4. **Verificar:** Vuelve a ejecutar el programa con los mismos datos del paso 1 para asegurar que el fallo ha desaparecido y prueba con otros datos nuevos para asegurar que no has roto nada más (efectos colaterales).



B. Técnicas de Depuración Básica

Para llevar a cabo el diagnóstico, existen diferentes técnicas según la complejidad del problema:

1. **Traza de Escritorio (Manual).** Es la técnica más básica y académica. Consiste en simular ser el ordenador utilizando papel y lápiz.

- Se crea una tabla con una columna para cada variable.
- Se recorre el algoritmo línea a línea y se van anotando los cambios de valor en la tabla.
- *Utilidad:* Es fundamental para estudiantes principiantes para entender la lógica de bucles y condicionales antes de usar herramientas automáticas.

2. **Depuración por trazas (Logging / Print debugging).** Consiste en insertar sentencias de impresión temporalmente en el código para ver "por dónde pasa" el programa y qué valores tiene.

- *En Java:* Escribir `System.out.println("Valor de x: " + x);` en puntos clave.
- *Ventaja:* Es rápido y fácil de implementar.
- *Desventaja:* Ensucia el código y hay que borrarlo todo al terminar. Si olvidas borrar uno, el usuario final verá mensajes técnicos extraños.

3. **Depuración Interactiva (Uso del IDE).** Es la forma profesional, utilizando los puntos de ruptura (breakpoints) que vimos en el punto 5.4 .

- Permite ver la **Pila de Llamadas (Stack Trace)**: Cuando ocurre una excepción (error de ejecución), Java imprime un rastro que te dice exactamente en qué línea explotó el programa y qué métodos se llamaron antes. Leer el *Stack Trace* es la habilidad número uno para detectar errores de ejecución.

  Importante sobre el Stack Trace: Cuando veas un mensaje de error rojo gigante en la consola, no te asustes. Busca la primera línea que haga referencia a tu propio código (a tus clases), y ahí estará el problema.

6.3 Calidad del código: legibilidad, modularidad y mantenibilidad

La calidad del software se mide por tres pilares fundamentales que aseguran que el programa no solo resuelva el problema hoy, sino que sobreviva al paso del tiempo.

A. Legibilidad (Code Readability)

Un principio famoso en programación dice: *"El código se lee muchas más veces de las que se escribe"*. La legibilidad es la facilidad con la que un humano puede entender lo que hace el código simplemente leyéndolo.

Para lograr una buena legibilidad en Java, seguimos estas reglas:

1. **Convenciones de Nombres:**

- Las variables y métodos deben tener nombres descriptivos. Evita `a`, `b`, `x`. Usa `salarioEmpleado`, `calcularImpuesto()`.
- En Java se usa **camelCase** (primera letra minúscula, siguientes palabras con mayúscula).

2. **Indentación y Formato:**

- El código debe estar correctamente tabulado para visualizar la jerarquía. El contenido de un `if` o un bucle debe estar desplazado a la derecha. Los IDEs lo hacen automáticamente.

3. **Comentarios útiles:**

- El código explica el *qué* y el *cómo*, los comentarios deben explicar el **porqué**.
- No comentes lo obvio (`i++ // Incremento i`), comenta la lógica compleja (`// Se aplica un descuento especial por ser familia numerosa`).
- *Requisito curricular:* La documentación es un criterio de evaluación explícito ("Se ha comentado y documentado el código").

B. Modularidad

La modularidad es la capacidad de dividir un sistema complejo en piezas más pequeñas, independientes y manejables (módulos, clases o métodos) .

- **Principio de Responsabilidad Única:** Cada clase o método debe hacer una sola cosa y hacerla bien.
    - *Mal ejemplo:* Un método `gestionarSistema()` que pide datos, calcula nóminas, imprime facturas y envía emails (Clase Dios).
    - *Buen ejemplo:* Un método `calcularNomina()`, otro `imprimirFactura()`, etc.
- **Reutilización:** Si divides bien el código, puedes reutilizar esas piezas en otros proyectos sin tener que copiar y pegar.
- **Aislamiento de errores:** Si falla la impresión, sabes que el error está en el módulo de impresión, no en el de cálculo matemático.

C. Mantenibilidad

Es la facilidad con la que el software puede ser modificado para corregir fallos, mejorar su rendimiento o adaptarse a nuevos requisitos.

Un código es mantenible si:

1. **Es fácil de cambiar:** Tocar una parte del código no rompe otras diez partes inesperadamente (bajo acoplamiento).
2. **Es fácil de probar:** Al estar modularizado, se pueden probar las piezas por separado.
3. **Es escalable:** Se pueden añadir nuevas funcionalidades sin tener que reescribir todo el sistema base.

   La deuda técnica: Escribir código rápido y sucio (sin calidad) genera "deuda técnica". Al principio avanzas rápido, pero a largo plazo, el coste de mantener ese código desordenado es tan alto que el proyecto se paraliza.

6.4 Introducción al aseguramiento de la calidad

El aseguramiento de la calidad no es "probar a ver si funciona" de forma aleatoria. Es una disciplina de la ingeniería de software que planifica y ejecuta pruebas rigurosas.

Tipos de Pruebas de Software (Testing)

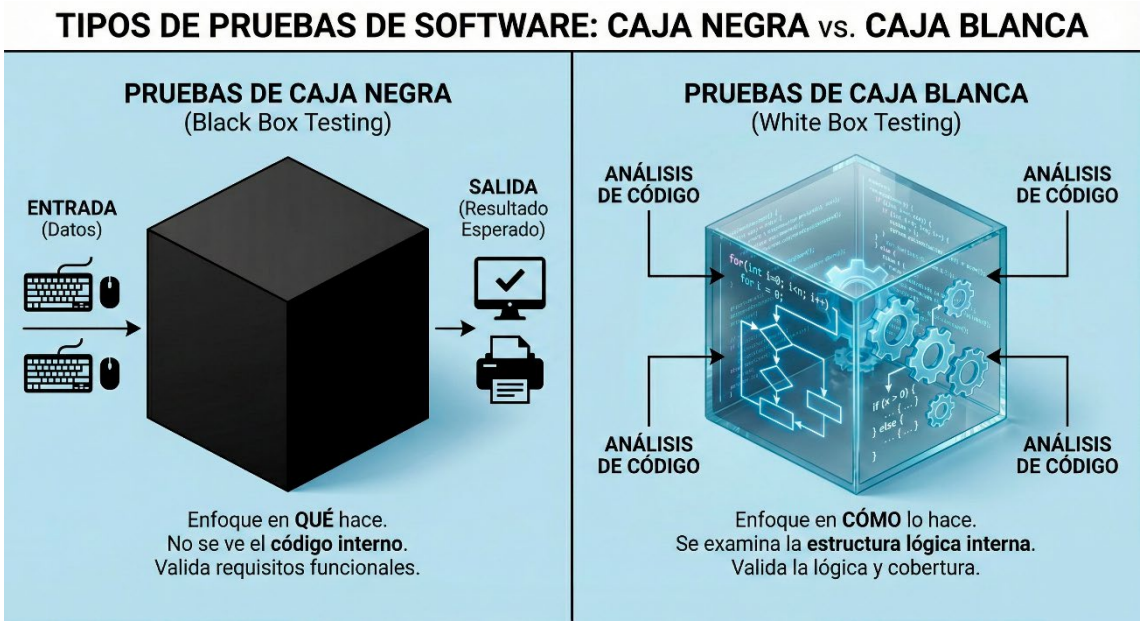
En esta fase inicial, es importante que distingas dos enfoques principales a la hora de verificar tu código Java:

1. **Pruebas de Caja Negra (Black Box Testing):**

- Se realizan sin mirar el código fuente. Nos centramos solo en las **entradas** y las **salidas**.
- El probador actúa como si fuera el usuario final. No le importa *cómo* está hecho el programa por dentro, solo le importa que si pulsa el botón "Guardar", el archivo se guarde.
- *Objetivo:* Validar que el programa cumple los requisitos funcionales.

2. **Pruebas de Caja Blanca (White Box Testing):**

- Se realizan mirando el código interno. El probador (que suele ser el propio programador) examina la estructura lógica.
- Se intenta asegurar que todas las líneas del código se ejecuten al menos una vez (cobertura de código) y que los bucles y condicionales estén bien construidos.
- *Objetivo:* Validar la lógica interna y la estructura.



El Plan de Pruebas y los Casos de Prueba

Para cumplir con la calidad, no basta con probar "un par de cosas". Se diseñan **Casos de Prueba** (Test Cases). Un caso de prueba es un documento o guion que define:

- **Entrada:** ¿Qué datos vamos a introducir? (Ej: Introducir edad `-5`).
- **Resultado Esperado:** ¿Qué debería pasar? (Ej: El programa debe mostrar error "Edad no válida").
- **Resultado Obtenido:** ¿Qué pasó realmente? (Ej: El programa aceptó el -5 y falló más tarde).

Consejo para el curso: Cuando entregues una práctica, diseña casos de prueba para los **casos límite** (bordes).

- Si tu programa pide una nota del 0 al 10, ¿qué pasa si metes un 0? ¿Y un 10? ¿Y un 11? ¿Y una letra? Un buen software debe aguantar todo eso sin romperse.

Automatización: JUnit

Finalmente, es vital mencionar que en la industria moderna las pruebas no se hacen a mano cada vez. Se automatizan.

En Java, el estándar es JUnit. Es una librería que permite escribir código que prueba tu propio código (Pruebas Unitarias).

- Tú escribes un test que dice: *"Comprueba que* `suma(2,2)` *da* `4`*"*.
- Cada vez que modificas el programa, ejecutas JUnit y él pasa miles de pruebas en segundos para asegurarte de que no has roto nada.

Resumen

1. Conceptos Base: ¿Qué es programar?

Programar no es solo escribir código; es resolver problemas.

- **Dato vs. Información:**
  - *Dato:* La materia prima (ej. "25", "Juan"). Por sí solo no dice nada.
  - *Información:* El dato procesado y con contexto (ej. "Juan tiene 25 años").
- **Algoritmo:** Es la receta. Una secuencia finita, ordenada y precisa de pasos para convertir datos de entrada en información de salida.
  - Se puede representar gráficamente (**Diagramas de Flujo**) o con texto estructurado (**Pseudocódigo**).

2. Paradigmas: ¿Cómo organizamos el pensamiento?

No existe una única forma de diseñar software. Dependiendo del enfoque, tenemos:

- **Imperativo:** Damos órdenes paso a paso cambiando el estado de la memoria.
- **Estructurado:** Ponemos orden al imperativo. Eliminamos el caos (GOTO) usando solo **Secuencia**, **Selección** (if) e **Iteración** (bucles). Es la base de la lógica interna de Java.
- **Orientado a Objetos (POO):** El rey de este curso.
  - En lugar de funciones sueltas, modelamos el mundo real.
  - **Clase:** El molde (ej. `Coche`).
  - **Objeto:** La copia real creada con el molde (ej. `MiFerrari`).
  - Unimos datos y comportamiento en una sola entidad (**Encapsulamiento**).

3. El Lenguaje: ¿Cómo funciona Java?

Java es un lenguaje de **Alto Nivel** (cercano al humano) que utiliza un sistema de traducción híbrido único para lograr su famosa portabilidad ("*Write Once, Run Anywhere*").

El ciclo de vida de tu código Java es:

1. **Código Fuente (`.java`):** Lo que tú escribes y entiendes.
2. **Compilador (`javac`):** Revisa la sintaxis. Si está bien, lo traduce.
3. **Bytecode (`.class`):** Un código intermedio, optimizado pero genérico. No es para la CPU, es para la JVM.
4. **Máquina Virtual (JVM):** El intérprete inteligente. Lee el Bytecode y lo traduce al instante al código máquina real de tu ordenador (Windows, Linux, Mac).

4. El Entorno de Trabajo (Las Herramientas)

Para ser productivo y profesional, necesitas más que un bloc de notas:

- **IDE (Entorno de Desarrollo Integrado):** Tu taller completo (Eclipse, IntelliJ, NetBeans).
    - Integra: Editor inteligente + Compilador + Depurador.
    - Te ayuda con autocompletado y detección de errores en tiempo real.
- **Control de Versiones (Git):** Tu red de seguridad. Guarda el historial de cambios para que puedas volver atrás si rompes algo y facilita el trabajo en equipo.
- **Herramientas de Build (Maven/Gradle):** Automatizan la "construcción" del proyecto y gestionan las librerías externas para que no tengas que descargarlas a mano.

5. Gestión de Errores y Calidad

El código perfecto no existe a la primera. Debes saber identificar qué falla:

Tipo de Error	Síntoma	Culpable Habitual	Solución
Sintaxis	El programa no arranca (línea roja en el IDE).	Faltan `;`, `}` o palabras mal escritas.	Corregir la gramática.
Ejecución (Runtime)	El programa se cierra de golpe (Crash/Excepción).	División por cero, leer archivos inexistentes, datos nulos.	Control de excepciones (`try-catch`).
Lógico	El programa funciona pero da resultados incorrectos.	Fallo en el diseño del algoritmo (sumar en vez de restar).	Usar el **Depurador** paso a paso.

Calidad del Software:

No basta con que funcione. El código debe ser:

- **Legible:** Buenos nombres de variables y formato limpio.
- **Modular:** Dividido en piezas pequeñas (funciones/clases) con responsabilidades claras.
- **Mantenible:** Fácil de modificar en el futuro sin romper todo el sistema.